

En el momento de calcular $O(\text{Time})$, se toman en cuenta las constantes que ocurren. Time

1

Términos matemáticos

Exponencial:

$$a) x^A \cdot x^B = x^{A+B}$$

$$b) x^A / x^B = x^{A-B}$$

$$c) (x^A)^B = x^{AB}$$

$$d) x^n + x^n = 2x^n$$

$$e) 2^n + 2^n = 2 \cdot 2^n = 2^{n+1}$$

Logarítmicas (generalmente $\log A = \log_2 A$):

Definición: Sea $x^A = B \rightarrow \log_x B = A$

Teoremas:

$$a) \log_A B = \frac{\log_c B}{\log_c A} ; A, B, c > 0 \text{ con } A \neq 1$$

$$b) \log AB = \log A + \log B ; A, B > 0$$

$$c) \log A/B = \log A - \log B$$

$$d) \log(A^B) = B \log A$$

$$e) \log x < x, \forall x > 0$$

Series: Geométricas:

$$a) \sum_{i=0}^N 2^i = 2^{N+1} - 1$$

$$b) \sum_{i=0}^N A^i = \frac{A^{N+1} - 1}{A - 1}$$

$$c) \sum_{i=0}^N A^i \leq \frac{1}{1-A} \quad \text{si } 0 < A < 1$$

Aritméticas:

$$a) \sum_{i=1}^N i = \frac{N(N+1)}{2} \approx \frac{N^2}{2}$$

$$b) \sum_{i=1}^N i^2 = \frac{N(N+1)(2N+1)}{6} \approx \frac{N^3}{3}$$

$$c) \sum_{i=1}^N i^K \approx \frac{N^{K+1}}{K+1}, K \neq -1$$

IMPORTANTE

$$\sum_{k=i}^j 1 = j - i + 1$$

Recursión:

función recursiva: una función definido en términos de ella misma.

Formada por:

① caso base: valor para el cual la función es conocida sin recurrir a la llamada recursiva.

② llamada recursiva: instancias en una recursión

- Reglas:
- a) Siempre debe existir un caso base (que se resuelve sin recursión)
 - b) Siempre se debe garantizar que se llegue al caso base
 - c) Asumir que todas las llamadas recursivas funcionen
 - d) Nunca duplicar trabajo mediante la resolución de la misma instancia de un problema en llamadas recursivas separadas.

Ejemplo:

```
public boolean paridad(int n) { // n ≥ 1
    if (n == 1)                // ①
        return false;         // ②
    else
        if (n == 0)            // ③
            return true;       // ④
        else
            return paridad(n / 2); // ⑤
}
```

Casos base: líneas ② y ③

Llamada recursiva: línea ⑤

Para empezar se plantea una función recursiva:

$$T(n) = \begin{cases} c & n=1; \\ c & n=0; \\ T(n/2) & n>1; \end{cases}$$

Función Recursiva

Todos los $i=1$

2

Bucles: while ($i \leq N$) {
 $i += h$
}

while ($i \leq N$) {
 $i = i * 2$
}

while ($i \leq N$) {
 $i = i * h$
}

while ($i \geq N$) {
 $i -= h$
}

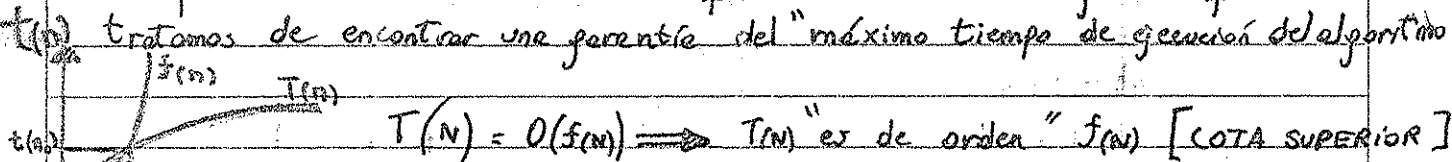
while ($i \geq N$) {
 $i = i / 2$ $\longrightarrow \log_2(N) + 1$

while ($i \geq N$) { $\longrightarrow \log_h(N) + 1$
 $i = i / h$
}

En estos bucles hay que prestar atención, específicamente, a la manipulación y transformación del índice ("i" en estos casos) que aparece en la verificación. El desarrollo de este índice nos indicará las repeticiones del bucle.

Definición Big-Oh: Sean las constantes positivas (c) y (n_0) tales que $T(n) \leq c \cdot f(n)$ cuando $n \geq n_0 \rightarrow T(n) = O(f(n))$.

Basicamente, lo que se trata de demostrar con el Big-Oh es que para cualquier función (en este caso nuestro tiempo de ejecución $T(n)$) siempre va a existir otra función que la acote. Esto significa que tratamos de encontrar una parentela del "máximo tiempo de ejecución del algoritmo".



Sin embargo, hay momentos en los que $f(n) < T(n)$ para algunos valores de n . Así, estamos comparando sus RATIOS de CRECIMIENTO RELATIVO.

Por lo tanto, al definir un punto constante (n_0) se garantiza que la función $c \cdot f(n)$ siempre será más grande (o, como mínimo, igual) que nuestro tiempo de ejecución $T(n)$.

• Otra forma de considerar los RATIOS de CRECIMIENTO RELATIVOS es viendo el valor de $\lim_{n \rightarrow \infty} \frac{T(n)}{f(n)}$:

- el límite es 0 $\rightarrow$ significa que $T(n) = o(f(n))$
- el límite es $c \neq 0 \rightarrow$ significa que $T(n) = \Theta(f(n))$
- el límite es $\infty \rightarrow f(n) = o(T(n))$

(Little-oh $\rightarrow T(n) = o(f(n))$ significa "el ratio de crecimiento de $T(n)$ es menor ($<$) que el de $f(n)$ ".)
(Theta $\rightarrow T(n) = \Theta(f(n))$ significa "el ratio de crecimiento de $T(n)$ es igual ($=$) que el de $f(n)$ ".)

Ratios típicos de crecimiento:

Reglas:

$f(n)$	nombre
c	constante
$\log(N)$	Logarítmico
$\log^2(N)$	Log-cuadrático
$N \cdot \log(N)$	
N^2	cuadrático
N^3	cúbico
2^N	exponencial

1) Sean $T_1(N) = O(f(N))$ and $T_2(N) = O(g(N))$ entonces:

- $T_1(N) + T_2(N) = O(f(N) + g(N))$. Menos formal sería $T_1(N) + T_2(N) = \max(O(f), O(g))$
- $T_1(N) \cdot T_2(N) = O(f(N) \cdot g(N))$

$$= \max(O(f), O(g))$$

2) Sea $T(n)$ un polinomio de grado $k \rightarrow T(n) = \Theta(N^k)$

3) Sea $\log^K N = O(N)$ p/ cualquier constante K

La habilidad de hacer un análisis genera una comprensión en el diseño eficiente de algoritmos. 3

Análisis: al analizar un algoritmo, el recurso más importante es el tiempo de ejecución.

Para esto consideremos:

• el tamaño de la entrada, N .

• $T_{prom}(N)$ y $T_{peor}(N)$, con significado "el caso promedio de tiempo" y "el peor tiempo" respectivamente.

* N siempre debe ser mayor o igual a 1.

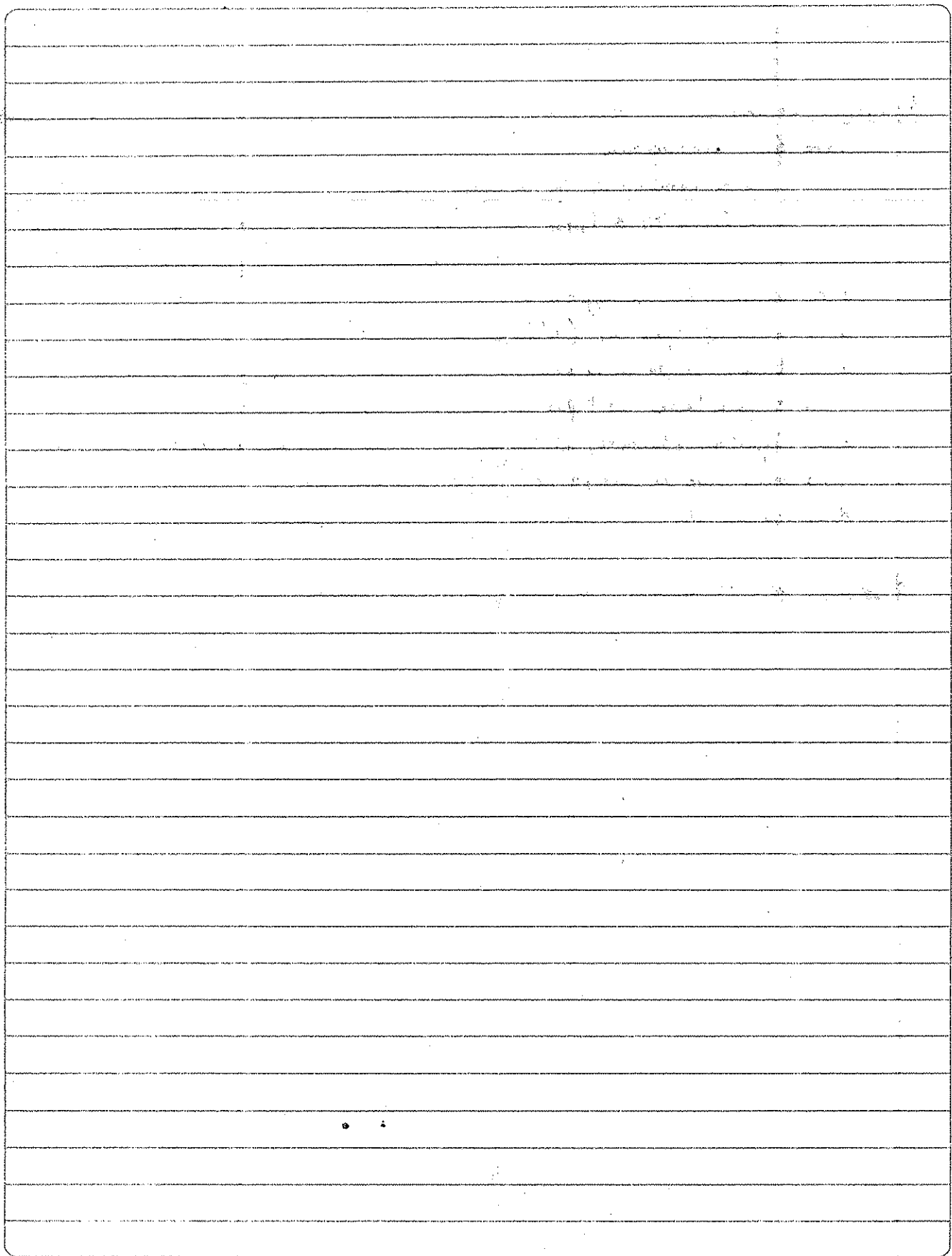
* $T_{prom}(N) \leq T_{peor}(N)$ (c/ tiempo depende del tamaño de la entrada).

* El tiempo buscado en el análisis es el $T_{peor}(N)$, ya que se establezca un límite para todas las tipos de entrada (inclusive una mala entrada).

* En algoritmos eficientes, el tiempo en tardar de leer los datos de entrada es mayor que el tiempo de resolución del algoritmo.

* Nunca se debe sub-estimar el T_{prom} de un programa.

⊗ Cálculo T_{prom} : en un algoritmo se debe discernir entre



public static int maxSubSum1(int [] a) {

int maxSum = 0;

// c₁

for (int i=0; i < a.length; i++)

// indice corre. N veces.

for (int j=i; j < a.length; j++)

{ int thisSum = 0;

// c₂

for (int k=i; k <= j; k++)

thisSum += a[k];

if (thisSum > maxSum)

maxSum = thisSum;

// c₃

}

return maxSum;

}

$$T(n) = \sum_{i=0}^{n-1} \left(\sum_{j=i}^{n-1} \left(c + \sum_{k=i}^j c \right) \right) = \sum_{i=0}^{n-1} \sum_{j=i}^{n-1} (j-i+1) =$$

$$= \sum_{i=0}^{n-1} (n-i)(n-i+1)$$

$$\sum_{j=i+1}^n j = \sum_{j=1}^{n-1} j = \frac{(n-i)(n-i+1)}{2} = \frac{1}{2} \left(n(n-i+1) - i(n-i+1) \right)$$

$$n^2 + n - in + i^2 - in - i$$

$$\frac{n^2}{2} + \frac{i^2}{2} + \frac{n-i}{2} - \frac{in}{2}$$

$$\frac{n(n+1)}{2} - \frac{i(i+1)}{2}$$

• public static void dos (int N) {

int i, j, x, y;

x = 0;

y = 0;

for (i = 1; i <= n-1; i++) {

if ((n % 2) == 1) {

for (j = 1; j <= i; j++)

{ x = x + 1;

for (j = 1; j <= i; j++)

{ y = y + 1;

}

}

}

(1)

(2)

(3)

(4)

(5) // Si el contador i empezare desde 0,

(6) // en la línea (5) se haria $\sum_{i=0}^N \rightarrow 3$ veces

(7)

(8)

(9)

(10)

0 1 2 3

$$\sum_{i=1}^{n-1} i = \frac{(n-1)(n+1)}{2}$$

• El peor de los casos seria que la entrada N sea impar, por la línea (6), ya que siempre se ejecutaría los bucles internos (7) y (9).

$$T(n) = \sum_{i=1}^{n-1} \left(\sum_{j=1}^i c + \sum_{j=1}^i c \right) = c \sum_{i=1}^{n-1} \sum_{j=1}^i 1 + c \sum_{i=1}^{n-1} \sum_{j=1}^i 1 =$$

$$= c \sum_{i=1}^{n-1} i + c \sum_{i=1}^{n-1} i = c[(n-1)(n)] + c \frac{(n-1)(n)}{2} =$$

no depende de "i" depende de "i"

$$= c[n^2 - n] + \frac{c}{2} [n^2 - n] = \frac{(n^2 - n)(c + \frac{c}{2})}{2} = \frac{cn^2 - cn}{2} + \frac{cn^2 - cn}{2} =$$

$$= \frac{cn^2}{2} \left(1 + \frac{1}{2}\right) - \frac{cn}{2} \left(1 + \frac{1}{2}\right) = \frac{3}{2} cn^2 - \frac{3}{2} cn$$

Sean constantes $c = 2$ y $n_0 = 2$

$$\frac{3}{2} c n^2 - \frac{3}{2} c n > 0$$

$$6c - 3c = 3c$$

$$c \cdot n^2 \geq \frac{3}{2} n^2 - \frac{3}{2} n \quad \forall n \geq n_0 \Rightarrow T(n) = O(n^2)$$

Resolución de $\Rightarrow \sum_{i=1}^{n-1} \sum_{j=i}^n 1 = \sum_{i=1}^{n-1} (n-i+1) = \sum_{i=1}^{n-1} n - \sum_{i=1}^{n-1} i + \sum_{i=1}^{n-1} 1 =$

$$= (n-1)(n) - \frac{(n-1)(n)}{2} + (n-1) = n^2 - n - \frac{n^2}{2} + \frac{n}{2} + n - 1 =$$

$$= \frac{1}{2} n^2 + \frac{3}{2} n - 1$$

rusares

```

• public static void tres (int n) { (1)
    int i, j, k, sum; (2)
    sum = 0; (3)
    for (i = 1; i <= n; i++) (4)
    {
        for (j = 1; j <= i * i; j++) (5)
        {
            for (k = 1; k <= j; k++) (6)
            {
                sum = sum + 1; (7)
            }
        }
    }
}

```

$$\begin{aligned}
 T(n) &= \sum_{i=1}^n \sum_{j=1}^{i^2} \sum_{k=1}^j c = c \sum_{i=1}^n \sum_{j=1}^{i^2} \sum_{k=1}^j 1 = c \sum_{i=1}^n \sum_{j=1}^{i^2} j = c \sum_{i=1}^n \frac{i^2(i^2+1)}{2} = \\
 &= \frac{c}{2} \sum_{i=1}^n (i^4 + i^2) = \frac{c}{2} \left(\sum_{i=1}^n i^4 + \sum_{i=1}^n i^2 \right) = \frac{c}{2} \left[\frac{n(n+1)(6n^3+9n^2+n-1)}{30} + \right. \\
 &\quad \left. + \frac{n(n+1)(2n+1)}{6} \right] = \frac{c}{2} \left[\frac{1}{30} (n^2+n)(6n^3+9n^2+n-1) + \frac{1}{6} (n^2+n)(2n+1) \right] = \\
 &= \frac{c}{2} \left[\frac{1}{30} (6n^5+9n^4+n^3-n^2+6n^4+9n^3+n^2-n) + \frac{1}{6} (2n^3+n^2+2n^2+n) \right] = \\
 &= \frac{c}{60} (6n^5+15n^4+10n^3-n) + \frac{c}{12} (2n^3+3n^2+n) = \text{Distribuímos las constantes...}
 \end{aligned}$$

$T(n) = O(n^5)$ ya que es un polinomio de grado 5 (REGLA DEL POLINOMIO)

```

int c = 1;           (1)
while (c < n) {       (2)
    algo de O(1);     (3)
    c = 2 * c;         (4)
}

```

con K = cantidad de ejecuciones while.

- Como el índice "c" va creciendo de a potencias de 2 $\Rightarrow c = 2^{K-1}$

$$T(n) = \sum_{c=1}^{\log(n)} 1 = \log(n) \rightarrow \boxed{T(n) = O(\log(n))}$$

ya que $\exists c=1$ y $n_0=1$ t.q. $c \cdot n \geq \frac{n}{2} \forall n \geq n_0$.

```

int x = 1;           (1)
for (int i = 1; i < n; i = i + 4) (2)
    for (int j = 1; j < n; j = j + |n/4|) (3)
        for (int k = 1; k < n; k = k * 2) (4)
            x = x + 1; (5)

```

$|n/4| \rightarrow$ división entera

- Hay que prestar atención a como van cambiando los índices

- En el bucle (2) siempre se ejecuta $\frac{n}{4} + 1$, porque el índice crece de a 4 y la primera vuelta siempre se hace

- El bucle (4), el índice "j" va creciendo en potencias de 2 así que hay $\lceil \log(n) + 1 \rceil$ repeticiones.

- El bucle (3) va creciendo como el bucle 2, ya que va aumentando de a $1/4$ partes de (n), y como la primera siempre se ejecuta $\Rightarrow$ hay $\frac{n}{4} + 1$ repeticiones.

$$a = b = \frac{n}{4} + 1 \text{ y } c = \log(n) + 1$$

$$T(n) = \sum_{i=1}^a \sum_{j=1}^b \sum_{k=1}^c c = c \sum_{i=1}^a \sum_{j=1}^b \sum_{k=1}^c 1 = c \sum_{i=1}^a \sum_{j=1}^b [\log(n) + 1] =$$

$$= c \sum_{i=1}^a \left(\frac{n}{4} \log(n) + \log(n) + \frac{n}{4} + 1 \right) = c \left(\frac{1}{4} \sum_{i=1}^a n \log(n) + \sum_{i=1}^a \log(n) + \frac{1}{4} \sum_{i=1}^a n + \sum_{i=1}^a 1 \right) =$$

$$= c \left(\frac{1}{4} \left[\frac{1}{4} n^2 \log(n) + n \log n \right] + \frac{n}{4} \log n + \log n + \frac{1}{4} \left[\frac{1}{4} n^2 + n \right] + \frac{n}{4} + 1 \right) =$$

$$= c \left(\frac{1}{16} n^2 \log(n) + \frac{1}{4} n \log n + \frac{1}{4} n \log n + \log n + \frac{1}{16} n^2 + \frac{1}{4} n + \frac{1}{4} n + 1 \right)$$

$$= c \left(\frac{1}{16} n^2 \log n + \frac{1}{4} n \log n + \log n + \frac{1}{16} n^2 + \frac{1}{2} n + 1 \right)$$

```

static public Integer rec1 (Integer n) {
    if (n <= 1)
        return 1;
    else
        return (rec1(n-1) + rec1(n-1));
}

```

(1) (2) → caso base (3) (4) (5) → Recursión

$$T(n) = \begin{cases} c & n \leq 1 \\ 2T(n-1) & n > 1 \end{cases}$$

Recursiones (K)

$K=1 \quad T(n) = 2T(n-1) \quad // \text{param: } n$
 $K=2 \quad T(n) = 2(2T(n-1-1)) = 4T(n-2) \quad // \text{param: } n-1$
 $K=3 \quad T(n) = 4(2T(n-2-1)) = 8T(n-3) \quad // \text{param: } n-2$
 $K=4 \quad T(n) = 8(2T(n-3-1)) = 16T(n-4) \quad // \text{param: } n-3$
 $\dots$
 $K=i \quad T(n) = 2^i T(n-i)$

Si $n-i \leq 1 \rightarrow i=n$. Luego, $T(n) = 2^n \cdot 1$.

Sean $c=1$ y $n_0=1 \rightarrow c \cdot 2^n \geq 2^n \quad \forall n \geq n_0 \Rightarrow \therefore T(n) = O(2^n)$

```

static public Integer rec2 (Integer n) {
    if (n <= 1)
        return 1;
    else
        return (2 * rec2(n-1));
}

```

(1) (2) (3) (4) (5)

- En la línea (5) hay una llamada recursiva, cuyo resultado se multiplica por 2 por ser algo constante.

$$T(n) = \begin{cases} c & , n \leq 1 \\ T(n-1) + d & , n > 1 \end{cases}$$

Recursiones (K)

$K=1, \text{ param: } n, \quad T(n) = T(n-1) + d$
 $K=2, \text{ param: } n-1, \quad T(n) = (T(n-1-1) + d) + d = T(n-2) + 2d$
 $K=3, \text{ param: } n-2, \quad T(n) = (T(n-2-1) + d) + 2d = T(n-3) + 3d$
 $\dots$

$K=i, \text{ param: } n-i-1, \quad T(n) = T(n-i) + i \cdot d$

Si $n-i = 1 \rightarrow i=n \rightarrow \text{Luego, } T(n) = n \cdot d + c$

Sean $c_1 = c$, $c_2 = d$ y $n_0 = 1$ entonces $(c+d)n \geq nd + c$. $\therefore T(n) = O(n)$

```

static public Integer rec3 (Integer n) {
    if (n == 0)
        return 0;
    else
        if (n == 1)
            return 1;
        else
            return (rec3(n-2) * rec3(n-2));
}

```

$$T(n) = \begin{cases} c_1 & , n = 0 \\ c_2 & , n = 1 \\ 2T(n-2) + c_2 & , n > 1 \end{cases}$$

Recurciones:

$$K=1, \text{ param: } n, T(n) = 2T(n-2) + c_2$$

$$K=2, \text{ param: } n-2, T(n) = 2(2T(n-2-2) + c_2) + c_2 = 2^2 T(n-4) + 2c_2$$

$$K=3, \text{ param: } n-4, T(n) = 2^2(2T(n-4-2) + c_2) + 2c_2 = 2^3 T(n-6) + 3c_2$$

$$K=i, T(n) = 2^i T(n-2i) + i \cdot c_2$$

$$\text{Cuando } n-2i = 1 \rightarrow i = \frac{n-1}{2} \text{ y } T(n) = 2^{(n-1)/2} + \frac{n-1}{2} \cdot c_2 =$$

$$\text{Sean } C = c_2 \text{ y } n_0 = 1 \text{ entonces } (2^{(n-1)/2} + \frac{n-1}{2} \cdot c_2) \leq C \cdot 2^n \quad \forall n \geq n_0 \quad \therefore T(n) = O(2^n)$$

```

static public int potencia - Rec (Integer x, Integer n) {
    Integer potencia;
    if (n == 0)
        potencia = 1;
    else
        if (n == 1)
            potencia = x;
        else {
            potencia = x;
            for (int i = 2; i <= n; i++)
                potencia = potencia * x;
        }
    return potencia;
}

```

Este no es un método Recursivo.

- El peor caso sera que se ejecute el "else" de la linea (8).

$$T(n) = d + \sum_{i=2}^n c = d + c \sum_{i=2}^n 1 = d + c(n-2+1) = d + c(n-1) = d + cn + c$$

Sean $c_1 = c$ y $n_0 = 1 \rightarrow c \cdot n \leq c_1 n \quad \forall n \geq n_0 \therefore T(n) = O(n)$

```

• static public Integer potencia_rec (Integer x, Integer n) {
    if (n == 0)                                     (1)
        return 1;                                  (2)
    else                                             (3)
        if (n == 1)                                 (4)
            return x;                               (5)
        else                                         (6)
            if ((n % 2) == 0)                         (7)
                return potencia_rec (x * x, n/2);    (8)
            else                                     (9)
                return potencia_rec (x * x, n/2) * x; (10)
}                                                    (11)

```

- Si la cantidad de entradas (n) es par, siempre se ejecuta la línea (8). Sino, la (10).

- Los casos (8) y (10) son similares, aunque la línea (10) tiene una constante más de ejecución.

$$T(n) = \begin{cases} c_1 & , n=0 \\ c_2 & , n=1 \\ T(n/2) + d & , n \geq 2 \end{cases}$$

$$K=1, \text{ param: } n \rightarrow T(n) = T(n/2) + d$$

$$K=2, \text{ param: } n/2 \rightarrow T(n) = [T(n/4) + d] + d = T(n/2^2) + 2d$$

$$K=3, \text{ param: } n/4 \rightarrow T(n) = [T(n/8) + d] + 2d = T(n/2^3) + 3d$$

$$K=i, \rightarrow T(n) = T(n/2^i) + i \cdot d$$

$$\text{Cuando } n/2^i = 1 \rightarrow n = 2^i \rightarrow \log n = \log 2^i \rightarrow i = \log n$$

$$\therefore T(n) = c_1 + \log(n) \cdot d$$

Sean $c = d$ y $n_0 = 1$ entonces $\log(n) \cdot d \leq \log(n) \cdot c \therefore T(n) = O(\log(n))$

```

1 public class Peral {
2     int [][] a = new int [n][n];
3     public void recu (int [][] a, int n) {
4         int [][] tmp = new int [n][n];
5         if (n > 1)
6             recu (tmp, n/2);
7         for (int i = 0; i < tmp.length; i++)
8             for (int j = 0; j < tmp.length; j++)
9                 tmp[i][j] = a[i][j] * tmp[i][j];
10        if (n > 1)
11            recu (a, n/2);
12    }

```

- El caso base es implícito en este caso, ya que se terminará la recursión cuando $n \leq 1$, en líneas (5) y (10). El análisis de esas líneas es una constante c_1 .

- Mientras la entrada (n) sea mayor a 1 se ejecutará * proceso:

• 2 recursiones (líneas 6 y 11).

• 2 bucles anidados (líneas 7 y 8).

$$T(n) = \begin{cases} c_1 & , n \leq 1 \\ 2T(n/2) + n^2 \cdot c_2 & , n > 1 \end{cases}$$

$$K=1, \text{ param: } (n), T(n) = 2T(n/2) + n^2 \cdot c_2$$

$$K=2, \text{ param: } (n/2), T(n) = 2(2T(n/2^2) + (n/2)^2 \cdot c_2) + n^2 \cdot c_2 = 2^2 T(n/2^2) + \frac{1}{2} n^2 \cdot c_2 + n^2 \cdot c_2$$

$$K=3, \text{ param: } (n/2^2), T(n) = 2^2(2T(n/2^3) + (n/2^2)^2 \cdot c_2) + \frac{1}{2} n^2 \cdot c_2 + n^2 \cdot c_2 =$$

$$= 2^3 T(n/2^3) + \frac{1}{4} n^2 \cdot c_2 + n^2 \cdot c_2 + \frac{1}{2} n^2 \cdot c_2 =$$

$$K=4, \text{ param: } (n/2^3), T(n) = 2^3(2T(n/2^4) + (\frac{1}{2^3} n)^2 \cdot c_2) + \frac{1}{4} n^2 \cdot c_2 + \frac{1}{2} n^2 \cdot c_2 + n^2 \cdot c_2 =$$

$$= 2^4 T(n/2^4) + \frac{1}{2^6} n^2 \cdot c_2 + \frac{1}{2^4} n^2 \cdot c_2 + \frac{1}{2^2} n^2 \cdot c_2 + n^2 \cdot c_2 =$$

$$= 2^4 T(n/2^4) + n^2 \cdot c_2 \cdot (1 + \frac{1}{2^2} + \frac{1}{2^4} + \frac{1}{2^6}) = \quad \quad \quad \frac{4}{3}$$

$$K=i, T(n) = 2^i \cdot T(n/2^i) + n^2 \cdot c_2 \left(\sum_{j=0}^i 2^{-2j} \right) = 2^i T(n/2^i) + n^2 \cdot c_2 \left(\frac{1}{1-2^{-2}} \right)$$

$$(*) \sum_{j=0}^n a^j \leq \frac{1}{1-a}, \text{ si } 0 < a < 1.$$

Cuando $n/2^i = 1 \rightarrow n = 2^i \rightarrow \log(n) = \log(2^i) \rightarrow \log(n) = i$ Entonces

$$T(n) \leq 2^{\log(n)} \cdot c_1 + \frac{4}{3} n^2 \cdot c_2 = n \cdot c_1 + \frac{4}{3} n^2 \cdot c_2$$

Soit $C = \frac{4}{3}C_2 + C_1$ y $n_0 = 1$, Δ Et valide

$$C \cdot n^2 \leq C_1 n + \frac{4}{3} C_2 n^2, \forall n \geq n_0$$

$$\boxed{\text{o' a } T(n) = O(n^2)}$$

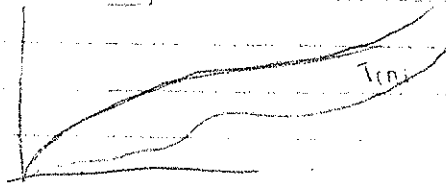
On a forme de justification

$$C_1 n \leq C_1 n^2 \quad \forall n \geq 1$$

$$\frac{4}{3} C_2 n^2 \leq \frac{4}{3} C_2 n^2 \quad \forall n \geq 1$$

$$C_1 n + \frac{4}{3} C_2 n^2 \leq \left(C_1 + \frac{4}{3} C_2 \right) n^2 \quad \forall n \geq 1$$

(H)



PREORDEN

```

preOrden (NodoBinario) {
    si (nodo != NULL) entonces
        preOrden (nodo.Izq);
        preOrden (nodo.Der);
    }

```

IN-ORDEN

```

inOrden (NodoBinario) {
    si (nodo != NULL) entonces
        inOrden (nodo.Izq);
        inOrden (nodo.Der);
    }

```

POST-ORDEN

```

postOrden (NodoBinario) {
    si (nodo != NULL) entonces
        postOrden (nodo.Izq);
        postOrden (nodo.Der);
    }

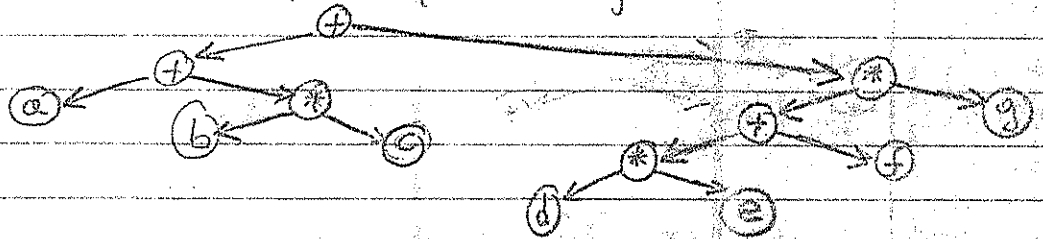
```

- APLICACIONES

- Árboles de expresión

Dada una expresión matemática simple, ésta se puede evaluar en un árbol binario. Sus hojas son OPERANDOS y los nodos internos contienen los OPERADORES BINARIOS (+, -, *, /) // OPERADORES UNARIOS (max, min, ...)

Podemos evaluar la expresión aplicándole alguno de los RECORRIDOS a la raíz.



① RECORRIDO INORDEN (raíz) = Expresión Infija = $(a + (b * c)) + (((d * e) + f) * g)$

② RECORRIDO PREORDEN (raíz) = Expresión Prefija = $+ + a * b c * + d e f g$

③ RECORRIDO POSTORDEN (raíz) = Expresión Postfija = $a b c * + d e * f + g * +$

- Por la definición recursiva de un AB, es común escribir rutinas recursivas.

- RECORRIDO POR NIVEL: utilizada en árboles generales, UTILIZANDO UNA COLA ADICIONAL

- Por NIVEL (T árbol)

Q = cola de nodos;

encolar(Q, T);

ImprimirRaíz(T);

mientras (la cola no esté vacía) hacer

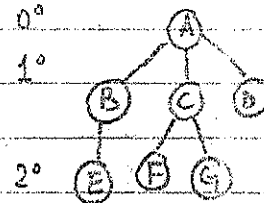
desencolar(Q, T);

L = lista de Hijos de Raíz(T)

por (c/ dato h en L) hacer //

encolar(Q, h)

Imprimir(h)



Por NIVEL (A) = A-B-C-D-E-F-G

- Esto operará en $O(n)$ ya que se busca pasar por todos los nodos sin repetir ninguno.

- Además, es un método no recursivo

Árbol Binario de búsqueda (ABB)

- La característica que lo distingue es que "para cada nodo, X , en el árbol, los valores de todas las ítems en su subárbol izquierdo son menores que el ítem en X , y los valores de todas los ítems del subárbol derecho son mayores que el ítem en X ." Esto es su PROPIEDAD DE ORDEN
- Debido a la propiedad de orden, sin fundamentos para realizar búsquedas, con $T(n) = O(\log n)$.
- La profundidad promedio de un ABB es de $O(\log n)$.
- Como todos los ítem de los árboles necesitan ser ordenados, es necesario que se puedan COMPARAR (es decir, que implementen la interfaz Comparable y sean comparados usando el método "compareTo").

IMPLEMENTACIÓN

```
class NodoBinario < T extends Comparable < T > > {  
    T item;  
    NodoBinario < T > Izquierda;  
    NodoBinario < T > Derecha;  
}
```

Algunas operaciones:

• #estoContenido: busca en el árbol si un elemento X se encuentra.

```
public estoContenido( elemento ) {  
    devolver estoContenido( elemento, raíz )  
}
```

```
privado estoContenido( elem, nodo ) {  
    si (nodo == null) entonces devolver FALSO; // no se encontró  
    si (nodo.item < elem) entonces devolver estoContenido( elem, nodo.derecha );  
    si (nodo.item > elem)  
        devolver estoContenido( elem, nodo.izquierda );  
    sino // si nodo.item == elem  
        devolver TRUE; // se encontró  
}
```

• #buscaring y #buscartMax: buscan el último nodo hacia la izquierda y derecha, respectivamente, en el ABB.

• Se puede resolver recursivo e iterativamente.

```
public buscaring() {  
    devolver buscarMin( raíz );  
}
```

```
privado buscarMin( nodo ) {
```

```
    si (nodo == null) entonces devolver NULL; // se fin si no es
```

Pila TAD:

• Una pila es una lista ^{TAD} con la restricción de que las inserciones y borrados pueden ser hechos en el TOPE de la pila; es decir, el final de la lista.

• Operaciones fundamentales (interpretar en Jere)

INPUT $\{ + \text{PUSH}(\langle T \rangle, \text{vaItem}) \rightarrow$ insertar en el final de la lista TAD.

OUTPUT $\{ + \text{RT} \rightarrow \text{POP}() \rightarrow$ remueve y devuelve el último elemento insertado.
 $\{ + \langle T \rangle \text{TOP}() \rightarrow$ devuelve, sin eliminar, el último elemento insertado.

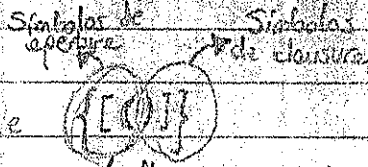
• Es una estructura LIFO (último en entrar, primero en salir)

• En una lista ENLAZADA, el TOPE se encuentra en el frente de la lista.

• Estas operaciones básicas deben garantizarse de que sean $O(1)$.

Aplicaciones:

- Evaluación de



• Se busca que el llave, corchete o paréntesis derecho tenga su contraparte izquierda.

• Procedimiento:

a) Crear una pila vacía

b) Leer caracteres hasta el final de la línea.

1) Si el carácter es un símbolo de apertura, se lo apila.

2) Si es un símbolo de clausura, se extrae un elemento y se verifica:

• si la pila está vacía $\rightarrow$ REPORTAR ERROR

• si el símbolo extraído no corresponde al de clausura $\rightarrow$ REPORTAR ERROR.

c) Al final de la línea, si la pila no está vacía se reporta error.

- Evaluación de expresiones POSTFIJAS, PREFIJAS e INFIJAS

- Para llamadas e métodos recursivos:

• Cuando se llaman a métodos o procedimientos dentro de un programa, es necesario guardar los registros usados por éste último para no perder sus valores.

• Cuando se realiza una LLAMADA A PROCEDIMIENTO $\rightarrow$ se apila un conjunto de registros correspondiente al método invocador.

• Cuando se realiza un RETURN $\rightarrow$ se desapila el último conjunto de registros y se los coloca en los registros de CPU correspondientes.

• Una pila en memoria, generalmente tiene su base en la dirección de memoria más alta, creciendo hacia direcciones inferiores.

TAD (Tipo abstracto de datos): es un conjunto de objetos junto con un conjunto de operaciones.

- Son abstracciones matemáticas (como CONJUNTOS y GRAFOS)
- El conjunto de operaciones ~~no~~ son implementadas en el TAD (sólo se encuentra la interfaz del objeto)

* Lista ^{General} TAD: Tiene la forma $A_0, A_1, A_2, \dots, A_N$; tiene tamaño N .

• Una lista vacía tiene tamaño 0.

• A_i es el sucesor de A_{i-1} (con $i < N$) y A_{i-1} antecede a A_i (con $i > 0$).

• Cada A_i es un objeto relacionado en el TAD con otros objetos de la misma clase.

- Implementaciones listas (ARREGLO)

• Con un ARREGLO de tamaño redimensionable.

• $T(n)$ de operaciones:

+ imprimir Elementos $\rightarrow T(n) = O(N)$

+ ver Elemento En Posición $\rightarrow T(n) = O(1)$

+ insertar Elemento / borrar Elemento

• peor caso = inserción en posición 0 $\rightarrow T(n) = O(N)$

• caso Promedio = inserción en mitad $\rightarrow T(n) = O(N)$

• mejor caso = inserción/borrado al final $\rightarrow T(n) = O(1)$

• Los elementos se encuentran contiguos en memoria.

* Lista Enlazada

• los elementos son una serie de NODOS, no necesariamente adyacentes en memoria. Cada uno contiene una referencia al próximo nodo



• Se mantiene un puntero al nodo primero y al último:

• $T(n)$ de operaciones:

+ imprimir Elementos $\rightarrow T(n) = O(N)$

+ ver Elemento En Posición $\rightarrow T(n) = O(N)$

+ inserción / borrado:

• mejor caso = insertar Al principio / Final / Borrar Al principio $\rightarrow T(n) = O(1)$

• peor caso = inserción o borrado en cualquier otra posición $\rightarrow T(n) = O(N)$

* Lista Doble Enlazada

• Cada nodo tiene un puntero al PRÓXIMO y al PREDECESOR

• Inserción / Borrado en cualquier posición $\rightarrow T(n) =$

Arboles

• Son una estructura de datos donde la mayoría de las operaciones es realizada en $O(\log N)$.

• DEFINICIÓN: Es una colección de N nodos.

- Existe un nodo (R) , llamado raíz.
- Está compuesto de 0 ó más subárboles: $S_1, S_2, S_3, \dots$
- Cada uno de ellos conectado con un vértice al nodo (R)
- Cada S_i es hijo de (R) , y (R) es padre de S_i
- Aquellos hijos que no sean padres (es decir, que no tienen hijos), se llaman HOJAS.
- Los hijos de un mismo padre son hermanos.

• DEFINICIÓN RECURSIVA:

- Existe (R) nodos conectados entre ellos mediante (R) vértices.
- Cada nodo, excepto la raíz, está conectado con su PAIRE y sus hijos, excepto los hijos.

• CARACTERÍSTICAS:

- CAMINO: ... desde $(n_1 \text{ a } n_k)$, es la secuencia de nodos $n_1, n_2, n_3, \dots, n_k$ conectados mediante $k-1$ aristas. n_1 es padre de n_2 , ..., con $2 \leq k \leq K$.
- LONGITUD: ... del camino $(n_1 - n_k)$, es la cantidad de aristas en el camino, es decir $n_1 - n_k$ tiene longitud $K-1$.
 - La longitud del camino $n_i - n_j$ (desde un nodo hacia el mismo) es 0.
 - Existe solo un camino desde la raíz a cualquier otro nodo.
- PROFUNDIDAD: ... de un nodo (n_i) , es la longitud del camino $n_0 - n_i$ (con n_0 como raíz).
 - La profundidad del árbol es igual a la profundidad de la hoja más profunda.
 - La profundidad de la raíz es 0.
- ALTURA: ... de un nodo (n_i) , es la longitud del camino más largo $n_i - n_j$, donde (n_j) representa una hoja del árbol.
 - La ALTURA del árbol es igual a la altura de la raíz.
 - Todas las hojas tienen Alturas 0.

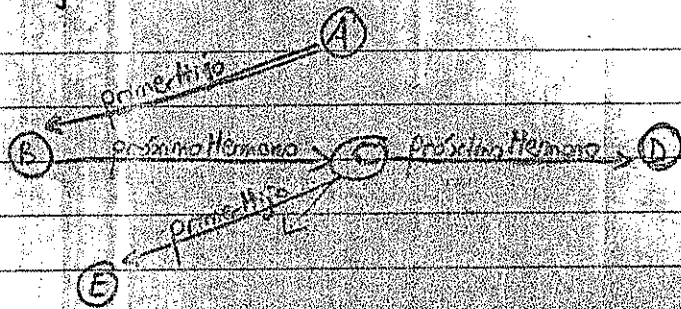
• Ver ANCESTRO PROPIO y DESCENDIENTE PROPIO,

• GRADO DEL ÁRBOL: es la cantidad máxima de hijos que puede llegar a tener un nodo del árbol.

• NIVEL DE UN NODO: longitud del único camino desde la raíz a un nodo.

- Un árbol general es aquel en el cual no se sabe con exactitud cuántas hijas podrá tener un nodo, es decir, que es un árbol de grado N.
- IMPLEMENTACIÓN (en Java)

```
class NodoGeneral <T>
{
    <T> elementoData;
    NodoGeneral <T> primerHijo;
    NodoGeneral <T> proximoHermano;
}
```



- APLICACIONES:

- Esta estructura se para administrar la jerarquía de directorios de varios sistemas operativos (UNIX, DOS).
- En UNIX, cada directorio es un nodo con una lista de todos sus subdirectorios y archivos (hijos). Los archivos son hojas.

Árboles binarios (AB)

- Son árboles de grado 2.
- Consiste en una raíz y 2 subárboles: S_L y S_R (donde cualquiera puede ser vacío).
- En promedio, la profundidad de un árbol binario es de $O(\sqrt{N})$.
- En el peor de los casos, el árbol puede convertirse en una lista, con una profundidad de $N-1$.
- Un árbol binario tiene repartidos N nodos y $N+1$ links en N nodos.
- IMPLEMENTACIÓN:

```
class NodoBinario <T>
{
    <T> elementoData;
    NodoBinario <T> hijoIzquierdo;
    NodoBinario <T> hijoDerecho;
}
```

- Recordados en la ejecución de los elementos de un árbol binario.

↳ EN PROFUNDIDAD

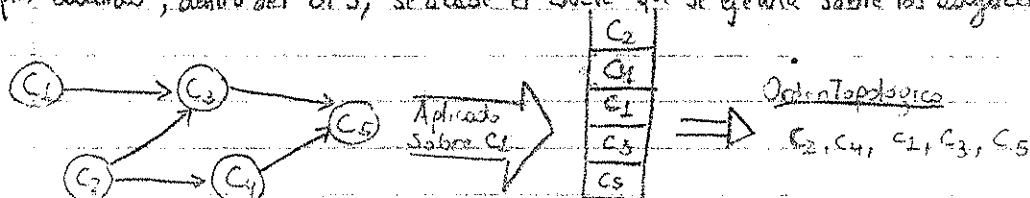


- PREORDEN: el trabajo sobre un nodo es realizado antes que sus hijos.
↳ ejemplo: PREORDEN (a) = a - b - c
- INORDEN: el trabajo se realice en el orden: hijo izquierdo - raíz - derecho.
↳ ejemplo: INORDEN (a) = b - a - c

3) DFS con RECORRIDO EN PROFUNDIDAD (DFS) →

- Necesitamos una estructura de pila P y arreglo Booleano $Visitados$ A .
- Con el DFS busca aquellos vértices que no pasaron adyacentes $\otimes$ procesar los voy apilando.
- El orden topológico puede determinarse por el orden en que voy apilando.
- El último elemento en la pila es el que más grados de entrada posee.
- Apila cuando, dentro del DFS, se acabe el bucle que se genera sobre los adyacentes.

Ejemplo:



Algoritmos de caminos de costo mínimo

- Dado un grafo pesado, donde $c/(v_i, v_j) \in |E|$ tiene un peso definida como C_{ij} :

el costo del camino $v_1, v_2, \dots, v_N$ es $\sum_{i=1}^{N-1} C_{i,i+1}$.

- Para grafos sin peso, el costo de un camino es la cantidad de aristas.

PROBLEMA: Dado un V inicial $\otimes$, se busca el camino menos costoso desde $\otimes$ al resto de los nodos.

→ Algoritmo de costo mínimo NO PESADO $O(V^2)$

- Desde el vértice inicial $\otimes$, hacemos un BFS, de forma tal que el costo de un camino hasta el nivel N , sea de $(N-1)$ aristas.
- Cada vértice mantiene cierta info en su estructura:
 - d_v : es el costo del camino mínimo (al comienzo desconocido, ∞).
 - p_v : indica el nodo previo del camino mínimo $S-v$ (al comienzo marcado como 0).
 - conocido: indica si ese nodo fue procesado previamente (comienza en FALSE).
- Cuando un vértice está "conocido", se garantiza de que $\neq$ camino menos costoso.
- Al procesar un vértice (v) , cada adyacente (w) se actualiza:
 - si $(d_w > d_v + C_{v,w}) \rightarrow d_w = d_v + C_{v,w}$
 - si se cumple lo anterior $\rightarrow p_w = v$.
- El camino se construye haciendo el camino inverso a través de (p_v) .

Ejemplo:

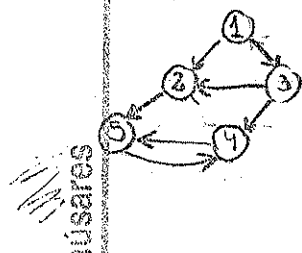
Dado el grafo G , sea $\otimes$ el vértice 1:

V	con	d_v	p_v
1	F	0	0
2	F	∞	0
3	F	∞	0
4	F	∞	0
5	F	∞	0

V	con	d_v	p_v
1	T	0	0
2	F	1	1
3	F	1	1
4	F	∞	0
5	F	∞	0

V	con	d_v	p_v
1	T	0	0
2	T	1	1
3	F	1	1
4	F	∞	0
5	F	2	2

V	con	d_v	p_v
1	T	0	0
2	T	1	1
3	T	1	1
4	F	2	3
5	F	2	2



agregar AListaDeSalida (v)

para c/w adyacentes a v

$GRADO_IN(w) = GRADO_IN(w) - 1$

CONTADOR incrementa en 1

//Comprobar por cual condiccion salio y:

// a) devolver la ListaDeSalida con el orden topologico

// b) lanzar una excepcion de ciclo encontrado

$$- T(|V|, |A|) = \sum_{i=1}^{|V|} (|V| + c + a_{v_i}) = |V|^2 + |V|c + |V| \overbrace{2}^{(|E|)}$$

$$\text{Luego } T(|V|, |E|) = O(|V|^2)$$

a_{v_i} es la cantidad de adyacentes de $v_i \in |V|$

2) RECORRIDO EN AMPLITUD CON UNA ESTRATEGIA ADICIONAL $O(|V| + |E|)$

- Se utiliza una cola Q o pila P para EVITAR el $O(|V|^2)$

- En Q o P se van guardando aquellas vertices con $GRADO_IN = 0$ a medida que se va trabajando

- Cuando procesamos v , decremos los $GRADO_IN$ de sus adyacentes w y verificamos si alguna queda con $GRADO_IN = 0$.

- Operación SORT-TOPOLÓGICA 2 (Arreglo GradosEntrada GI)

crearCola(Q)

Tomar v de $GRADO_IN = 0$, si $\nexists v$ lanzar excepcion No hay Cola // $O(|V|)$

encolar(Q, v)

mientras (colaNoVacía) hacer

desencolar(Q, v)

marcar a v como visitado con $GRADO_IN(v) = -1$

agregar AListaDeSalida (v) =

para c/w adyacentes a v hacer

$GRADO_IN(w)$ reducir en 1

si ($GRADO_IN(w) = 0$) entonces

encolar(Q, w)

//Comprobar si la cola está vacía y:

// a) devolver la ListaDeSalida si la está

// b) lanzar excepcion de ciclo si no está vacía

$$- T(|V|, |A|) = \sum_{i=1}^{|V|} (c + a_{v_i}) + |V| = |V| + |E| + |V| = 2|V| + |E|$$

$$\text{si } T(|V|, |A|) = O(|V| + |A|)$$

sino si (nodo.izquierda es null) entonces devolver nodo;
 devolver buscarMin (nodo.izquierda);
 }

• # insertar : • se inserta un nuevo elemento X en el ABB, manteniendo el orden.

- Hay que evitar duplicados, por lo que el detector este caso, se tendria que ignorar la insercion o actualizar un contador de frecuencia.
- los duplicados hacen que el arbol sea muy profundo, si no se los evita.

```

Publico INSERTAR (elemInsertar) {
    INSERTAR (elemInsertar, raíz);
}
  
```

```

PRIVATE INSERTAR (elemInsertar, nodo) {
    si (nodo = null) de i:
        devolver nuevaRaíz (elemInsertar);
    si (nodo.elem < elemInsertar)
        nodo.derecho = INSERTAR (elemInsertar, nodo.derecho);
    si (nodo.elem > elemInsertar)
        nodo.izquierda = INSERTAR (elemInsertar, nodo.izquierda);
    sino
        se actualiza el contador; //... de duplicados.
    devolver nodo;
}
  
```

• # REMOVE : Al eliminar un nodo, se debe actualizar ^{previamente} su padre el cual apuntara a un nuevo nodo en el lugar del que se borrara. Este nuevo nodo es el más izquierdo del subárbol derecho correspondiente al nodo a eliminar.

```

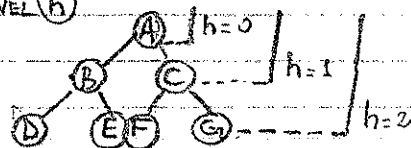
PRIVADO REMOVE (elemBorrar, nodo)
    si (nodo = null) devolver nodo; // no se encontró el elemento.
    si (nodo.elem < elemBorrar) entonces
        nodo.derecho = REMOVE (elemBorrar, nodo.derecho);
    si (nodo.elem > elemBorrar)
        nodo.izquierda = REMOVE (elemBorrar, nodo.izquierda);
    sino si (nodo tiene 2 hijas) // además es el nodo que se busca...
        buscar menor nodo derecho y reemplazar los valores del
        nodo a borrar por el buscado; luego, borrar el nodo buscado.
    // también verificar en el caso que sólo tenga 1 hijo el nodo borrar.
    devolver el nodo con los valores reemplazados.
  
```

Siempre se remueve de las hojas.

Propiedades de algunos Árboles:

- Árbol Lleno: un árbol (T) de altura $= h$, y grado $= K$, es LENO si: c/ nodo tiene (K) hijos y TODAS las hojas estén en un NIVEL (h)

→ Ejemplo: árbol binario LENO de $h=2$.



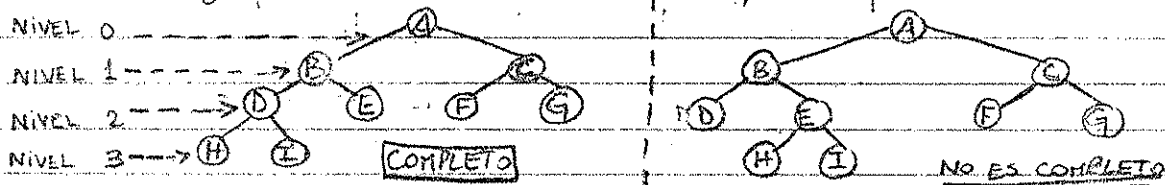
• Un árbol LENO tiene SIEMPRE:

$$\sum_{i=0}^h K^i = \frac{K^{h+1} - 1}{K - 1} \quad \text{NODOS}$$

→ Serie geométrica de razón $= K$

- Árbol Completo: c/ nodo tiene (K) hijos, en un árbol de altura $[h]$, se considera COMPLETO si los nodos en el último nivel h se van insertando de izquierda a derecha.

→ Ejemplo: sea un árbol binario de $h=3$, es completo si:



• Cantidad de nodos (N) de un árbol completo de grado (K) y altura (h) es:

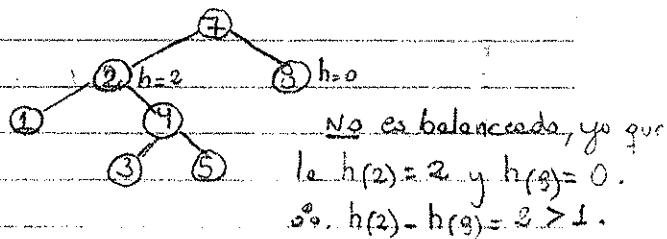
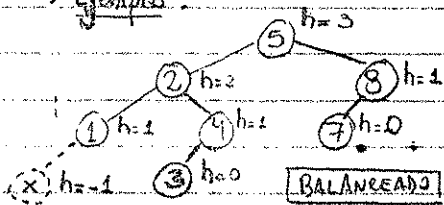
$$\frac{K^h + (K-2)}{K-1} < N < \frac{K^{h+1} - 1}{K-1}$$

→ N está acotada por las series geométricas de razón $= K-1$ y razón $= K$.

cantidad árbol hasta nivel $(h-1)$ $< N <$ cantidad árbol lleno hasta nivel (h)

- Árbol Balanceado: un árbol es BALANCEADO si para c/ nodo (N) se cumple que las diferencias de altura de cada subárbol de (N) , diffieren en 1 como máximo.

→ Ejemplo:



Árbol AVL

- Es un ABB que cumple con la propiedad de un árbol balanceado.
- Al insertar o remover un nodo, se debe garantizar:
 - 1º) Mantener la propiedad de orden (menores a izquierda, mayores a derecha).
 - 2º) Mantener el balance (REBALANCEANDO si es necesario).
 - 3º) Modificar la altura de un nodo cuando se es necesario.
- C/ nodo AVL mantiene información sobre su altura.
- La altura de un árbol AVL con N nodos, es de aprox. $\log N$.
- Todas las operaciones pueden ser llevadas en $O(\log N)$, a excepción de la inserción y la ELIMINACIÓN (no lógica).

• OPERACIÓN INSERCIÓN:

- Al insertar se debe actualizar toda la información de balance (altura) del camino desde la raíz hasta el nodo insertado, ya que solo los nodos de este camino tienen sus subárboles balanceados.

- La inserción puede violar la propiedad de balance en el AVL. Para arreglarlo se debe subir hacia arriba por el camino, encontrar el nodo desbalanceado y aplicar una ROTACIÓN, según el caso:

a) INSERCIÓN con desbalance externo:

* inserción izquierda - izquierda

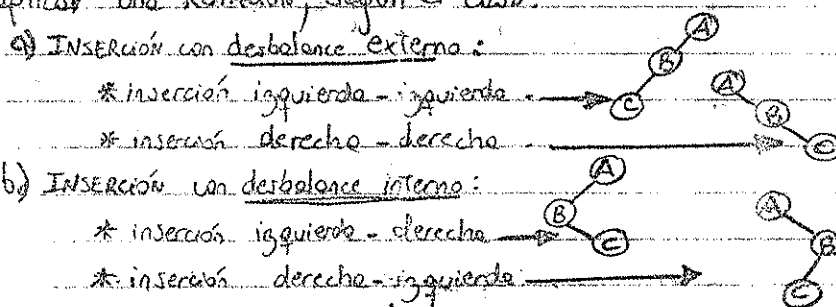
* inserción derecha - derecha

b) INSERCIÓN con desbalance interno:

* inserción izquierda - derecha

* inserción derecha - izquierda

cuando la diferencia de alturas
e/ 2 subárboles
es igual 2, se detecta
un desbalance.

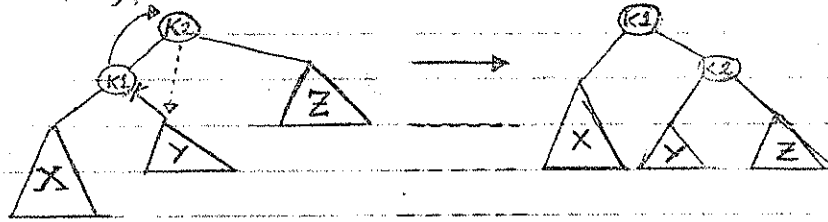


En todos los casos el nodo desbalanceado es el A, y el insertado es el C.

- ROTACIONES: al realinearlos sobre el nodo desbalanceado, se asegura que el AVL seguirá balanceado.

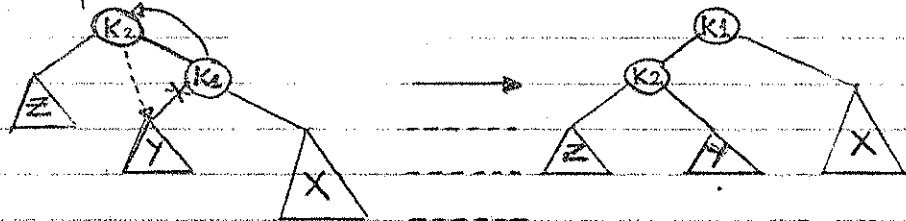
• rotación Simple:

- Simple Izquierda



- * $K2$ se encuentra desbalanceado, ya que la diferencia e/ los subárboles X y Z es 2.
- * X sube un nivel, Z desciende otra, mientras que Y se mantiene en el suyo. Además, $K1$ se convierte en la nueva raíz, e Y pasa a ser subárbol izquierdo de $K2$; mientras que éste, pasa a ser hijo derecho de $K1$.

- Simple Derecha



- Pseudocódigo

rotación Simple Izquierda (Nodo AVL K2)

1. <nuevo Nodo AVL> K1 = K2.izquierda;

2. K2.izquierda = K1.derecha;

3. K2.derecha = K2;

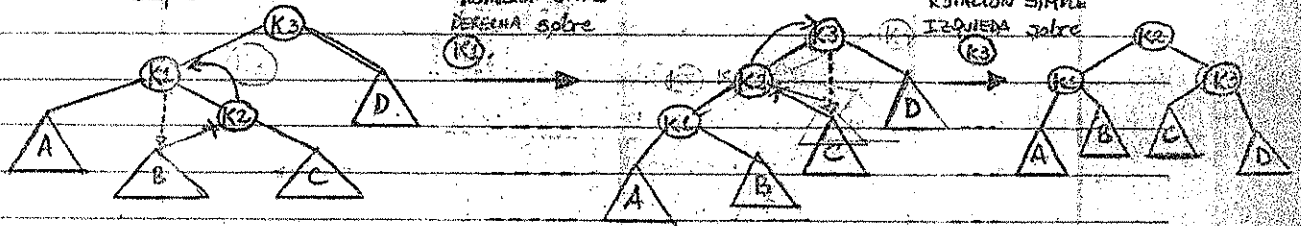
4. $Altura(K2) = \max(Altura(K2.izquierda), Altura(K2.derecha))$

5. // Idem con $Altura(K1)$...

6. devolver K2; // K2 es la nueva raíz del subárbol balanceado.

• Rotación Doble: Se basa en realizar dos rotaciones simples en los nodos que participan.

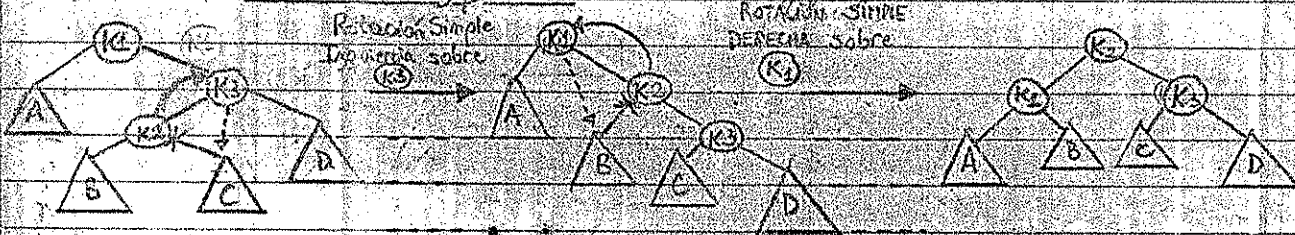
- Doble Izquierda - Derecha:



* K3 se encuentra desbalanceado, ya que la altura de K3 = $1\frac{1}{2} > \text{altura de D} = 1$

* Para solucionarlo, hay que forzar a que K1 sea hijo izquierdo de K2, y luego, que K3 sea hijo derecho de K2, cual pasará a ser la nueva raíz.

- Doble Derecha - Izquierda



- Pseudocódigo:

rotación Doble Izquierda (Nodo AVL K3)

K3.izquierda = rotación Simple Derecha (K3.izquierda);

devolver rotación Simple Izquierda (K3);

- IMPLEMENTACIÓN del NODO AVL: similar al NODO BINARIO, a excepción de que mantiene un campo correspondiente a la ALTURA del nodo.

- INSERTAR: "X" es el elemento a insertar.

```

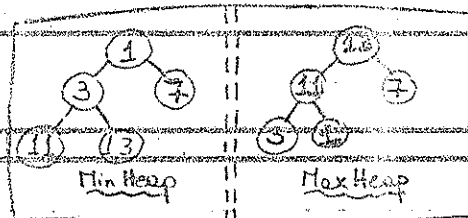
privado INSERTAR (X, nodo) {
    si (el "nodo" es nulo) devolver un NuevoNodoAVL (con X); // lugar correcto
                                                                para insertar
                                                                nueva hoja.
    si (X < nodo.elemento)
    {
        nodo.izquierda = INSERTAR (X, nodo.izquierda);
        si (altura (nodo.izquierda) - altura (nodo.derecha) = 2)
        {
            si (X < nodo.izquierda.elemento) // se verifica que rotación hacer.
                nodo = ROTACIÓN SIMPLE Izquierda (nodo);
            sino
                nodo = ROTACIÓN Doble Izquierda (nodo);
        }
    }
    sino si (X > nodo.elemento)
    {
        nodo.derecha = INSERTAR (X, nodo.derecha);
        si (altura (nodo.derecha) - altura (nodo.izquierda) = 2)
        {
            si (X > nodo.derecha.elemento) // se verifica que rotación hacer
                nodo = ROTACIÓN SIMPLE Derecha (nodo);
            sino
                nodo = ROTACIÓN Doble Derecha (nodo);
        }
    }
    sino
    {
        ; // se evitan los duplicados y no se hace nada.
    }
}

```

ACTUALIZAR ALTURA (nodo) // ...
 devolver nodo;

- Banner :

húsares



- HEAP (cola de prioridades):

* Es una estructura de datos que cumple con 2 propiedades:

- Propiedad ESTRUCTURAL: una HEAP es un ÁRBOL BINARIO COMPLETO
- Propiedad de ORDEN HEAP:

- MinHeap: el padre de cada nodo (excepto la raíz) tiene una clave menor que sus hijos.

- MaxHeap: el padre de cada nodo (excepto la raíz) tiene una clave mayor que sus hijos.

Con esta propiedad, siempre se asegura que el menor/mayor nodo SIEMPRE está en la raíz.

Por su simplicidad, una HEAP se puede implementar con un vector (de tamaño redimensionable), donde cada elemento se ubica en (suponiendo el nodo x en la posición " i "): :

- el padre (de x) en posición $\text{mod}(i/2)$ en el vector.
- el hijo Izquierdo (de x) en pos. $(2*i)$ en el vector.
- el hijo Derecho (de x) en pos. $(2*i) + 1$ en el vector.

EJEMPLO:

		X_p		X				X_{HI}	X_{HD}	
1	2	3	4	5	6	7	8	9	10	

$$\begin{aligned} \text{pos}(X) = 4 &\rightarrow \text{pos}(X_{\text{padre}}) = \lfloor 4/2 \rfloor = 2 & \text{pos}(X_{HI}) = 4*2 = 8 \\ \text{pos}(X_{HD}) &= (4*2) + 1 = 9 \end{aligned}$$

* Operaciones Básicas:

• INSERTAR: Siempre se inserta en el hijo izquierdo de una hoja en el último nivel (si el árbol está lleno) o en el próximo hijo libre de un nodo (si el árbol no está lleno). Al insertar se debe garantizar que se mantenga la propiedad de orden, mediante el proceso de: PERCOLATE-UP (filtrado hacia arriba).

• Máx/Min Remove: El elemento mayor/menor siempre se encuentra en la raíz, por lo que para eliminar el máx/min tenemos que BORRAR LA RAÍZ y REEMPLAZARLO por el último nodo ubicado en el último nivel. Al hacer este, se debe seguir garantizando la propiedad de orden, por lo que se debe hacer un PERCOLATE-DOWN (filtrado hacia abajo).

- Es una operación de $O(1)$

• FILTADOS

- PERCOLATE-UP: se trata de llevar un nodo, de niveles superiores, hacia algún lugar en un nivel inferior de la estructura, hasta que se cumpla la propiedad de orden.

* En este método siempre se va comparando y reemplazando un nodo con el padre.

- PERCOLATE-DOWN: se trata de llevar un nodo en niveles inferiores (más bajos)

- a su lugar correcto en algún nivel superior (más alto) hasta cumplir la propiedad de orden.
- En este método se va reemplazando un nodo padre por el menor/mayor de sus hijos.

PSEUDOCÓDIGO: (para una MinHeap)

- INSERTAR:

pública INSERTAR (elem)

verificar que el arreglo tenga capacidad, sino REDIMENSIONARLO.

posNueva = arregloTamano + 1;

while (posNueva > 1 AND elem > arreglo [posNueva / 2]) do { PERCOLATE_UP

arreglo [posNueva] = arreglo [posNueva / 2],

posNueva = posNueva / 2,

arreglo [posNueva] = elem

- MinRemove:

pública MinRemove ()

Verificar si es un árbol vacío;

elementoMinimo = arreglo [1]

arreglo [1] = arreglo [tamaño - 1] // Intercambio el último elemento con el 1°

percolateDown (1); // filtra el primer elemento hacia abajo

devolver elementoMinimo;

privado percolateDown (pos)

temp = arreglo [pos]; // en el (temp) se encuentre el elemento a FILTRAR

while (pos * 2 <= tamañoHeap) do

posHijo = pos * 2;

si (posHijo < tamañoHeap y arreglo [posHijo + 1] < arreglo [posHijo])

posHijo = posHijo + 1; // me quedo con la posición del Hijo con menor valor...

si (arreglo [posHijo] < temp)

arreglo [pos] = arreglo [posHijo]; // se intercambia con el padre si el hijo es menor.

sino

• Salir de Bucle;

pos = posHijo; // Si el padre sigue siendo mayor que el hijo, lo sigo FILTRANDO ABAJO.

arreglo [pos] = temp; // aquí ya encontré la posición correcta en la HEAP p/ el elem.

- Algunas $T(n)$:

- En Listas Enlazadas: INSERTAR en el lugar correcto es de $O(N)$.
 - BORRAR Mínimo (al principio o al final, manteniendo punteros) es de $O(1)$.
- En ABE: INSERTAR es de $O(\log N)$.
 - BORRAR Mínimo en el extremo Inguierdo. El borrado va a ser proporcional a la altura del árbol. En promedio, es de $O(\log N)$.
- En AVL: INSERTAR es de $O(\log N)$.
 - BORRAR Mínimo es de $O(\log N)$.
- En HEAP: INSERTAR en el peor de los casos (insertar un Nuevo Mínimo Raíz) es de $O(\log N)$.
 - ENCONTRAR Mínimo es de $O(1)$, ya que siempre se encuentra en el tope en una MinHeap.

- Otras Operaciones HEAP

- DECREMENTAR CLAVE: $DecrementeClave(p, \Delta)$ reduce el valor del nodo en la posición "p" con una cantidad Δ . Para arreglar la propiedad de orden se realiza $PERCOLATE_UP(p)$. Es de $O(\log N)$.
 - Uso: en S.O., los administradores pueden sobre la prioridad de ejecución de sus procesos.
- INCREMENTAR CLAVE: $IncrementarClave(p, \Delta)$ incrementa el valor de un nodo en la posición "p" con una cantidad positiva Δ . Para arreglar la propiedad de orden se realiza $PERCOLATE_DOWN(p)$. Es de $O(\log N)$.
 - Uso: en muchos S.O., los planificadores reducen la prioridad del proceso que está consumiendo tiempo excesivo de uso a la CPU.
- DELETE: $delete(p)$ remueve el nodo en la posición "p". Esto es hecho mediante un $DECREMENTAR CLAVE(p, \infty)$ (intentando llevar el nodo a la raíz) y mediante un $MinRemove()$. Es de $O(\log N)$.
 - Uso: Cuando un proceso es forzado a finalizar bruscamente éste debe ser removido de la cola de prioridad.
- CONSTRUIR HEAP: Dada una colección "C" de elementos, se construye a partir de ésta una cola de prioridad HEAP.
 - Tras insertando todos los x elementos desde C en la HEAP, serán en Orden:
 - $O(N)$ en el mejor de los casos, donde c/i insertará es de $O(1)$.
 - $O(N \log N)$ en el peor de los casos, donde c/i insertará es de $O(\log N)$.
 - Adeptando el siguiente método para CONSTRUIR HEAP, se garantiza un $O(N)$ en la operación:
 - 1) Volcar los elementos en la HEAP sin respetar la propiedad de ORDEN.
 - 2) Comenzando por el antepenúltimo nivel (nodo en la posición $T/2$, donde T es el tamaño de la HEAP), hacer $PERCOLATE_DOWN(T/2 - i)$, con $i = 0, 1, \dots, T/2$.

* Mediante este método, se deben hacer 2 comparaciones por nodo:

- 1) Comparar los hijos y ver el menor de ellos - } percolate-down.
- 2) Comparar el nodo con el menor ANTERIOR - } → así por el hijo que se baja.

* Para saber cuánto podría llegar a llegar la cantidad de comparaciones en el peor de los casos, se debe calcular la suma de altura de todos los nodos, ya que en el peor de los casos sería que un nodo INTERNO desciende a una hoja por el filtrado.

→ Entonces la SUMA DE LAS ALTURAS en un árbol binario completo sería:

$$\sum_{h=0}^h \sum_{i=1}^{2^h} 1 = \sum_{i=1}^{2^h} h = h \cdot 2^h$$

$$\boxed{2^{h+1} - 1 - (h+1)}$$

Prueba: Un árbol completo en i nodos de altura h , 2 nodos de altura $h-1$, 2^2 nodos de altura $h-2$, ..., 2^i nodos de altura $h-i$. Luego, la suma de las alturas es:

$$S = \sum_{i=0}^h 2^i (h-i)$$

$$S = h + 2(h-1) + 4(h-2) + \dots + 2^{h-1}(1) + 2^h(0)$$

Multiplicando $\times 2$ obtenemos:

$$2S = 2h + 4(h-1) + 8(h-2) + \dots + 2^h(1)$$

$$S + S = 2h + 4(h-1) + 8(h-2) + \dots + 2^h(1)$$

$$S = (2h + 4(h-1) + 8(h-2) + \dots + 2^h(1)) - S$$

Restando los términos correspondientes (p.e., $2h - 2(h-1) = 2$, $4(h-1) - 4(h-2) = 4$, ...):

$$S = -h + 2 + 4 + 8 + \dots + 2^{h-2} + 2^{h-1} + 2^h + 1 - 1 = 0$$

$$\boxed{\sum_{i=0}^h 2^i (h-i) = \frac{2^{h+1} - 1}{2} - (h+1)}$$

Se ve fácilmente de razón 2.

→ Como un árbol completo tiene entre 2^h y 2^{h+1} nodos, la suma se demuestra que es de $O(N)$, con N = número de nodos. → el construir HEAP es lineal.

* HEAPSORT: dado un conjunto de elementos sin orden, éste se puede ordenar (SORT) - de forma creciente o decreciente - utilizando en una HEAP, siendo éste operación de $O(N \log N)$.

- Con este método se evita la DUPLICACIÓN del espacio causado por el uso de una 2da estructura en donde se encuentran los elementos ordenados.

Método:

1º) Dado un vector con los elementos, éste se convertirá en una MaxHeap o una MinHeap, que empezará por la posición 0.

2º) Realizará Máx/Min Remove por cada elemento.

3º) Cada X que voy borrando, en vez de ponerlo en otro vector, lo coloco en la última posición y el último lo coloco en la primera posición, disminuyendo en 1 el tamaño de la HEAP.

4º) Realizar un Percolate-down () sobre el primer elemento.

- Con una MaxHeap $\rightarrow$ la ordenación será de Menor a mayor.
- MinHeap $\rightarrow$ la ordenación será de Mayor a menor.

Código en Java

```

HeapSort Descendiente (T extends Comparable<T>[] a) {
    <T> tmp;
    for (int i = a.length/2; i >= 1; i--)
        percolateDown(a, i, a.length);
    Construye Heap.
    Este bucle mide el T(n)
    que es de  $O(n \log(n))$ .
    for (int i = a.length; i > 1; i--)
        tmp = a[i]; a[i] = a[1]; a[1] = tmp; deleteMax(a, 1, i-1);
}

```

// percolateDown (heap, posición, tamañoHeap); //

- El problema de la selección del K-ésimo elemento MAYOR

1º Solución:

- Construyo una MaxHeap con los (N) elementos de entrada.
 - Voy haciendo sucesivas borradas.
 - Al K-ésimo borrada, obtengo el K-ésimo máximo.
- Es de $T(n) = K \cdot \log n \rightarrow O(N \cdot \log N)$

Pseudocódigo:

```

Selección K-Maxima (maxHeap H)
{
    i = 0; // mantiene el n° de borradas
    while (i <= k) do
    {
        deleteMax(H, x);
        i++;
    }
    return x;
}

```

2º Solución:

- Creo una MinHeap (S) con K elementos de la entrada (N) , desde (Como el K-ésimo elemento es el menor en (S) - llamado S_K - se encuentra en la raíz)
- Cada elemento x de los $(N-K)$ restantes, serán comparados con la raíz S_K .

c) Si $X > S_k$, ento. los hago deletar() para borrar S_k .

d) Luego, inserto X y lo filtro hacia arriba.

Al finalizar de procesar los $(N-K)$ elementos restantes, tengo al K -ésimo en la raíz.

$T(N)$

+ crear Heap con K elementos $\rightarrow O(K)$

+ procesar $(N-K)$ elementos restantes $\rightarrow O(1) \text{ c/v } \left. \vphantom{\begin{matrix} \\ \end{matrix}} \right\} (N-K) \log K$
 $\hookrightarrow$ percolate up de c/ restante $\rightarrow O(\log K)$

$$T(n) = O(K + (N-K) \log K) = O(N \log N)$$

Pseudocódigo

seleccionar (MinHeap de K : H , $(N-K)$ restantes R)

 por c/ x en R hacer:

 si $(x > S_K)$ entonces

 deleteMin(H);

 Insertar(H , x); // donde se hace un percolate up(x)

 return S_K ($j=1$) // es la raíz de la Heap.

GRAFO

Definiciones:

- GRAFO: consiste en un conjunto de vértices V y un conjunto de aristas E .

$$G = (V, E)$$

(nodos)

- VÉRTICES: son los elementos en donde se encuentran los datos consistentes de la estructura.

- ARISTA: cada una es un par (v, w) , donde $v, w \in V$.

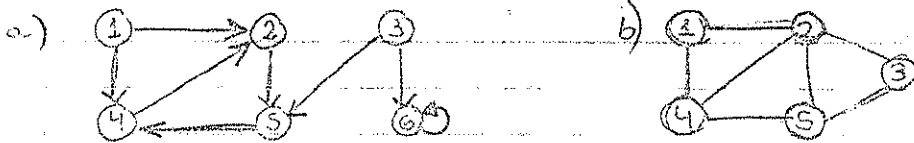
- CAMINO: es una secuencia de vértices $w_1, w_2, w_3, \dots, w_N$ t.q. $(w_i, w_{i+1}) \in E$ para $1 \leq i < N$.

- LONGITUD (de camino): número de aristas en un camino, igual a $N-1$, para el camino $w_1, \dots, w_N$.

• La longitud de un camino de un vértice w_i a sí mismo es de 1, si existe $(w_i, w_i) \in E$. En caso contrario, la longitud es 0.

- Camino Simple: camino en el que todos sus vértices son distintos, excepto quizás el primero y el último.

Ejemplos:



Grafo dirigido $G(V, E)$

$$V = \{1, 2, 3, 4, 5, 6\}$$

$$E = \{(1, 2), (1, 4), (2, 5), (3, 5), (4, 5), (5, 6), (6, 6)\}$$

Grafo No dirigido $G(V, E)$

$$V = \{1, 2, 3, 4, 5, 6\}$$

$$E = \{\{1, 2\}, \{1, 4\}, \{2, 3\}, \{2, 5\}, \{3, 5\}, \{4, 5\}, \{5, 6\}\}$$

- GRAFO DIRIGIDO: es un grafo cuyas aristas son pares ordenados, es decir, que tienen una dirección explícita. Son conocidos también como DIGRAFOS.

• Arista $\equiv$ par ordenado (u, v) : $u \rightarrow v$

- GRAFO NO DIRIGIDO: es un grafo cuyas aristas son no ordenados (simétricos).

• Arista $\equiv$ par no ordenado $\{u, v\}$ con $u \neq v$: $u - v$

- En realidad subyace un grafo con aristas bidireccionales.

$$u \rightleftarrows v \equiv u - v$$

- CICLO: camino simple desde $v_1, v_2, \dots, v_k$ t.q. $v_1 = v_k$.

Ej: $\langle 2, 5, 4, 2 \rangle$ es un ciclo de longitud 3 en el ejemplo a).

→ BUCLE: es un ciclo de longitud 1. Ej: $\langle 6, 6 \rangle$ es ciclo de long. 1 en a).

- GRAFO ACICLO: grafo sin ciclos, es decir, grafo en que todos caminos $v_1, \dots, v_k$ se cumple que $v_i \neq v_k$, $\forall i, k \in \mathbb{Z}$. Llamado DAG (Directed Acyclic graph).

- GRAFAS PESADOS: son aquellas en las que cada arista tiene asociado un peso o valor.



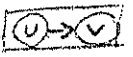
• Términos

- adyacencia: v es adyacente a u si $\exists (u,v) \in E$

• en grafos no dirigidos, $(u,v) \in E$ incide en los nodos u, v .

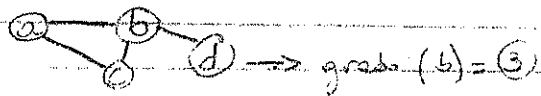


• en digrafos, $(u,v) \in E$ incide en v y le incide proviene de u .



- grado:

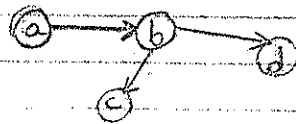
• en No dirigidos, el grado de un vértice es el número de aristas que inciden en él.



• en digrafos, el grado del vértice será la suma de los grados de entrada y de salida

- grado de entrada: n° de aristas que inciden en él.

- grado de salida: n° de aristas que parte desde él.



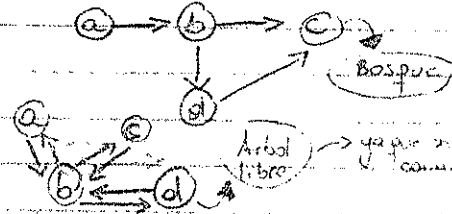
$$\text{grado-in}(b) = 2$$

$$\text{grado-out}(b) = 1$$

$$\text{grado}(d) = 1$$

- grado de un grafo: MÁXIMO grado de sus vértices.

- BOSQUE: es un grafo sin ciclos.



- ÁRBOL LIBRE: es un bosque conexo.

- Un árbol es un árbol libre en el que un nodo se ha designado como raíz.

RECORRIDOS sobre el grafo :

- Recorrido en profundidad (DFS) $\rightarrow$ [para digrafos y grafos no dirigidos]

- Debemos disponer de un arreglo booleano adicional, llamado 'ARREGLO DE VISITADOS', y nos permite evitar los ciclos.

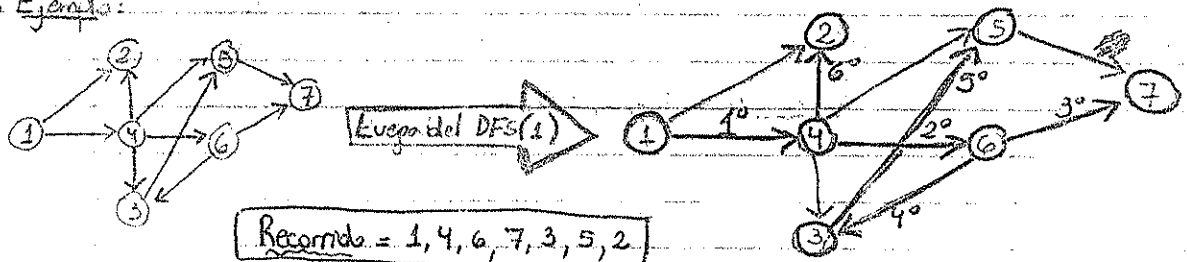
• MÉTODO:

- > Partir de un vértice aleatorio (V)
- > Por c/ nodo nuevo visitado, EXPLORAR cada camino que salga de él.
- > Hasta que no se haya finalizado de explorar una de las caminos no se comienza con el siguiente.
- > Un camino deja de explorarse cuando se llega a un vértice ya visitado.
- > Si quedan vértices no visitados (por ej. $U, X, \dots$) luego de hacer el recorrido sobre V , repetir el proceso sobre los faltantes. (Ésto ocurre en grafos de ramificación)

• ESQUEMA RECURSIVO: Dado $G = (V, E)$

- 1- Marcar, en el arreglo de visitados, todos los vértices como NO VISITADOS (en FALSO).
- 2- Elegir vértice (U) como punto de partida.
- 3- Marcar (U) como visitado.
- 4- $\forall$ V adyacente a (U), $(U, V) \in E$, si (V) no ha sido visitado, repetir recursivamente (2) y (3) para (V).
- 5- Verificar si faltan vértices por procesar.

• Ejemplo:



• PSEUDO CÓDIGO

operación DFS (v : Vértice)

marcar [v] := visitado

para cada vértice (w) adyacente a (v) hacer

si marcar [w] = no visitado entonces

DFS (w)

finPara

operación main_dfs ()

var marca : array Booleano de Visitados

para c/ posición (i) hacer

┌ marcar [i] := no visitado

para c/ posición (v) hacer

┌ si marcar [v] = no visitado entonces

└ DFS (nodo v)

REPRESENTACIONES

• Matriz Adyacente: Tenemos un arreglo bidireccional $a[V][V]$, en donde guardaremos información referente sobre las adyacencias entre vértices.

Para un DIGRADO: por $c/(u,v) \in E$, ponemos $a[u][v]$ en true si (u,v) son adyacentes. Ponemos falso en caso contrario.

Para un digrafo pesado: por $c/(u,v) \in E$, ponemos $a[u][v]$ con el valor del peso correspondiente si es que existe adyacencia. Caso contrario, usamos un centinela (∞ por ejemplo) para indicar la No adyacencia.

• Costo ESPACIAL: $O(|V|^2)$

- Pros: usar esta representación es apropiada para grafos densos ($|E| \approx |V| \times |V|$) donde la mayoría de las celdas en la matriz contendrán información relevante.

• Consultar si (u,v) pertenece a E es constante $\rightarrow T(|V|, |E|) = O(1)$

- Contras: Para grafos que no son densos, se puede desperdiciar mucho espacio ya que no hay demasiadas adyacencias.

• Lista de adyacencias

- Para $c/\text{vértice } (v)$ mantenemos de todas las vértices adyacentes.

- El espacio requerido así es lineal en el grado: $O(|E| + |V|)$

- Si $(u,v) \in E$, entonces se entenderá:

a. en DIGRAFOS: poner a (v) en la lista de adyacencias de (u)

b. en No DIGRAFOS: la misma que lo anterior, además de poner a (u) en la lista de adyacencias de (v) .

- En grafos no dirigidos, las adyacencias ocupan doble espacio.

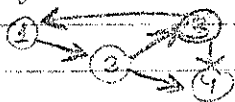
- Por lo tanto:

• Si G es dirigido $\rightarrow$ la suma de la longitudes de las listas de adyacencias será $|E|$.

• Si G es no dirigido $\rightarrow$ la suma será $2|E|$.

- La búsqueda de adyacentes será lineal $\rightarrow O(|E| + |V|)$.

Dado $G = \{1, 2, 3, 4\} \rightarrow$



Matriz de Adyacencia

	1	2	3	4
1	F	T	F	F
2	F	F	T	T
3	T	F	F	T
4	F	F	F	F

Lista de adyacencias

1	$\rightarrow$ 2
2	$\rightarrow$ 3 $\rightarrow$ 4
3	$\rightarrow$ 1 $\rightarrow$ 4
4	$\rightarrow$ //

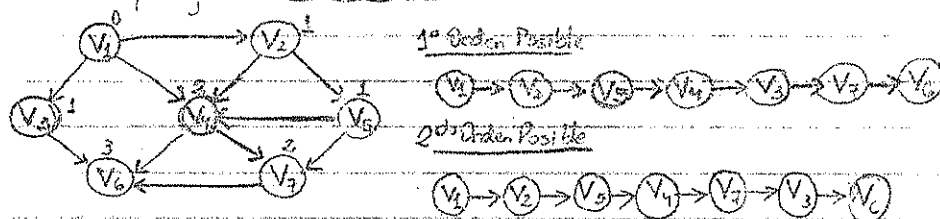
para cada posición (j) hacer
 si $noVisitado[j] = \text{NoVisitado}$ entonces
 L BFS (nodo j)

$$T(|V|, |E|) = O(|V| + |E|)$$

- SORT TOPOLOGICO:

- La ordenación topológica es una permutación $V_1, V_2, V_3, \dots, V_M$ de los vértices, tal que si $(V_i, V_j) \in E$, $V_i \neq V_j$, entonces V_i precede a V_j en la permutación.
- Como $V_i \neq V_j$, entonces una ordenación topológica EVITA los ciclos.
- El SORT TOPOLOGICO se realiza sobre DIGRAFOS ACICLICOS.
- La ordenación topológica no es única.

Ejemplo:



- Para realizar el SORT TOPOLOGICO, debe ir tratando progresivamente los vértices INDEPENDIENTES, es decir, aquellos cuyos grados in son 0.
- Por c/ vértice trabajado debe reducir los grados in de sus adyacentes.
- Para realizar el orden, NECESITAMOS una estructura arreglo con los GRADO-IN de c/ vértice (GRADO-IN = grado de entrada).
- Se llama GRAFO DE PREREQUISITOS a un grafo que cumple:
 "Dados, $(U, V) \in E$, no puedo trabajar con V si antes no lo hice con U ."

IMPLEMENTACIONES SORT:

1) Con un RECORRIDO EN ANCHURA ($O(|V|^2)$)

- Para encontrar a los (V) , $V \in V$, cuyo GRADO-IN = 0, debo recorrer linealmente el arreglo que mantiene los grados de entrada.

- operación SORT-TOPOLOGICA-1 (arreglo GradoEntrada GRADO-IN)

ciclo = enFalso

contador = 1

mientras (contador $\leq$ $|V|$ y noHayCiclo) hacer

tomar un (V) tal que $\text{GRADO-IN}(V) = 0$ // $O(|V|)$

si $\neq (V)$ entonces ciclo = enTrue

sino

marcar a (V) como visitado con $\text{GRADO-IN}(V) = -1$.

$$T(E, V) = O(V + E)$$

- Recorrido en amplitud o por niveles (BFS)

- para realizar esto se necesita

- 1 cola para mantener los nodos adyacentes.
- 1 arreglo booleano de visitados.

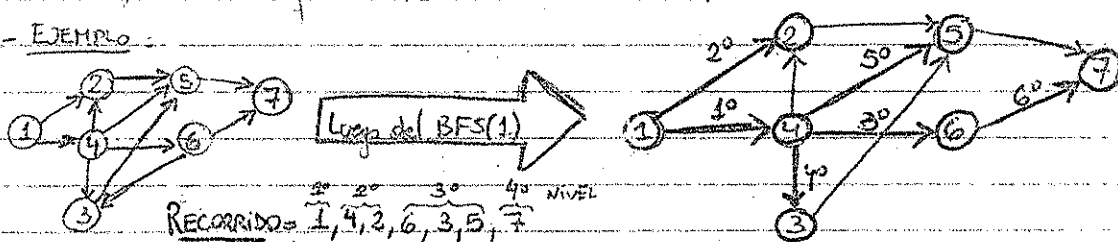
- MÉTODO:

- > Partir de algún vértice (u) , visitar (u) y después, visitar c/u de los vértices adyacentes a (u) .
- > Repetir el proceso para c/ vértice adyacente a (u) , siguiendo el orden en que fueron visitados.

- ESQUEMA ITERATIVO

- 1º) Inicializar el arreglo de visitados. Todos los vértices como No visitados.
- 2º) Encolamos un vértice (v) determinado.
- 3º) Marcamos a (v) como visitado.
- 4º) desencolamos (v) y lo tratamos.
- 5º) Encolamos los adyacentes de (v) que no hayan sido visitados y los marcamos como visitados.
- 6º) Seguimos con c/ (v) en la cola aplicando (4) y (5) hasta que la cola se vacíe.
- 7º) Verificamos si quedan vértices No visitados.

- EJEMPLO



- PSEUDOCÓDIGO

operación BFS (G, V)

marcar $[v] = \text{Visitado}$

encolar (v, v)

mientras $(\text{cola} \neq \text{Vacia}(Q))$ hacer

desencolar (q, v)

tratar (v)

para cada (w) adyacente a (v) hacer

• Si $\text{marcar}[w] = \text{No Visitado}$ entonces

marcar $[w] = \text{Visitado}$

encolar (Q, w)

main: operación bfs (i)

var $\text{marca} := \text{array Booleano De Visitados}$

para cada posición (i) hacer

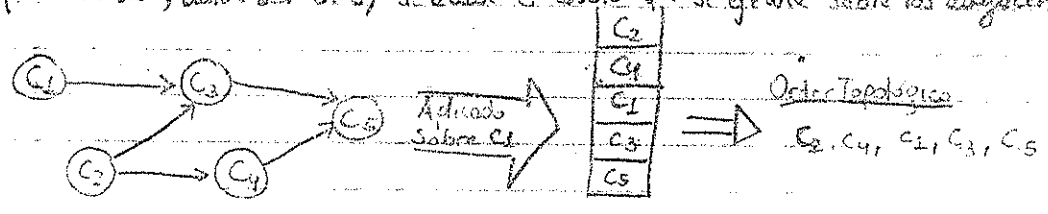
marcar $[i] := \text{No Visitado}$

visitas

3) SORT CON RECORRIDO EN PROFUNDIDAD (DFS) →

- Necesitamos una estructura de pila P y un array Booleano de Visitados A .
- Con el DFS buscamos los vértices que no pasaron adyacentes $\otimes$ ^{primero} los voy visitando.
- El orden topológico puede determinarse por el orden en que voy visitando.
- El último elemento en la pila es el que más grados de entrada posee.
- Apilando, dentro del DFS, se acaba el bucle que se ejecuta sobre los adyacentes.

Ejemplo:



Algoritmos de caminos de costo mínimo

- Dado un grafo ponderado, donde $c/(v_i, v_j) \in |E|$ tiene un peso definido como C_{ij} :
- el costo del camino $v_1, v_2, \dots, v_N$ es $\sum_{i=1}^{N-1} (C_{i,i+1})$.

- Para grafos sin peso, el costo de un camino es la cantidad de aristas.

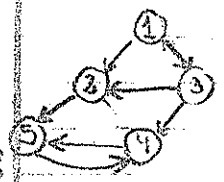
Problema: Dado un V inicial $\otimes$, se busca el camino menos costoso desde $\otimes$ al resto de los nodos.

Algoritmo de costo mínimo NO PESADO $[O(V^2)]$

- Desde el vértice inicial $\otimes$, hacemos un BFS, de forma tal que el costo de un camino hasta el nivel N , sea de (N) aristas.
- Cada vértice mantiene cierta info en su estructura:
 - d_v : es el costo del camino mínimo (al comienzo desconocido, ∞).
 - p_v : indica el nodo previo del camino mínimo $S \rightarrow v$ (al comienzo marcado como 0).
 - conocido: índice si ese nodo fue procesado previamente (comienza en FALSE).
- Cuando un vértice está "conocido", se garantiza de que $\exists$ camino menos costoso.
- Al procesar un vértice (v) , cada adyacente (w) se actualiza:
 - Si $(d_w > d_v + C_{v,w}) \rightarrow d_w = d_v + C_{v,w}$
 - Si se cumple lo anterior $\rightarrow p_w = v$.
- El camino se construye haciendo el camino inverso a través de (p_v) .

Ejemplo:

Dado el grafo G , sea $\otimes$ el vértice 1:



V	Con	d_v	p_v
1	F	0	0
2	F	∞	0
3	F	∞	0
4	F	∞	0
5	F	∞	0

V	Con	d_v	p_v
1	T	0	0
2	F	1	1
3	F	1	1
4	F	∞	0
5	F	∞	0

V	Con	d_v	p_v
1	T	0	0
2	T	1	1
3	F	1	1
4	F	∞	0
5	F	2	2

V	Con	d_v	p_v
1	T	0	0
2	T	1	1
3	T	1	1
4	F	2	3
5	F	2	2

húsaes

- Procesar el vértice 4 y 5 no cambia la última tabla.
- El camino mínimo desde 1 a 4 (usando el p_v) es: $1 \rightarrow 3 \rightarrow 4$, siendo $p_4 = 3$ y $p_3 = 1$. Se puede hacer mediante un método recursivo.

* PSEUDOCÓDIGO:

```

PROCEDURE sinPeso (Vértice s) {
    para c/ vértice (v) hacer
        |
        |   v.dist = ∞;
        |   v.conocido = falso;
        |
        |   s.dist = 0 → dist Actual
    bucle ① ← DESDE ① HASTA la CANT. VERTICES hacer (dist Actual ++ )
    bucle ② ← por c/ vértice (v) hacer
        |
        |   si ( ! v.conocido && v.dist = dist Actual ) entonces
        |       |
        |       |   v.conocido = TRUE;
        |       |   para c/ adyacente (w) a (v) hacer
        |       |       |
        |       |       |   si ( w.dist = ∞ ) entonces
        |       |       |       |
        |       |       |       |   w.dist = dist Actual;
        |       |       |       |   w.previo = v;
        |       |
        |       |   }
        |   }
    }

```

- El tiempo de ejecución es de $O(|V|^2)$ por los bucles anidados.

MEJORAMIENTO

- Se puede usar una cola para guardar los adyacentes.
- El campo "conocido" no se usa, ya que una vez procesado no puede entrar en la cola de vuelta.

* PSEUDOCÓDIGO

```

PROCEDURE con_min_cola (Vértice s) {
    cola Q;
    vértices v, w;
    encolar (s, Q);
    bucle 1 → |V| veces ← mientras: no vacío (Q) hacer
        |
        |   desencolar (v, Q);
        |   para c/ (w) adyacente a (v) hacer → bucle 2 → |E| veces
        |       |
        |       |   si ( w.dist = ∞ ) entonces
        |       |       |
        |       |       |   w.previo = v;
        |       |       |   w.dist = v.dist + 1;
        |       |       |   encolar (w, Q);
        |       |
        |       |   }
        |   }
    }

```

- El tiempo de ejecución es $O(|V| + |E|)$.

- PSEUDO CÓDIGO (sin minHeap)

```

Dijkstra ( $G, w, s$ ) { //  $w$  = adjacencia,  $s$  = origen
(1)   para  $\forall$  vértice  $v \in |V|$  hacer
(2)   |    $d_v = \infty$ ;  $p_v = 0$ ;
(3)   |    $d_s = 0$ ;
(4)   |   para  $\forall$  vértice  $v \in |V|$  hacer
(5)   |   |   Vértice  $u$  = vértice de menor distancia ();
(6)   |   |   Marcar a  $u$  como conocido;
(7)   |   |   para  $\forall$  vértice  $w \in |V|$  adyacente a  $u$  hacer
(8)   |   |   |   si ( $w$  no está conocido) entonces
(9)   |   |   |   |   si ( $d_w > d_u + C_{u,w}$ ) entonces
(10)  |   |   |   |   |    $d_w = d_u + C_{u,w}$ ;
(11)  |   |   |   |   |    $p_w = u$ ;
(12)  |   |   |   |   |   {Reorganizar HEAP, si es que se usa}
}
    
```

- $T(n) = O(|V|^2)$

- el bucle (4) se ejecuta p/ todas las vértices $\rightarrow |V|$ iteraciones
- La operación vértice de menor distancia (5) es $O(|V|)$ y dada que se realiza $|V|$ veces $\rightarrow$ el costo total de la operación (5) es $O(|V|^2)$.
- El bucle (7) se ejecuta para los adyacentes de $\forall$ vértice. El número TOTAL de iteraciones será la cantidad de aristas del grafo $\rightarrow |E|$ iteraciones

• $T(|V|, |E|) = \sum_{i=1}^{|V|} (|V| + a_{u_i}) + |V| + |V|^2 + |E| + |V|$; donde a_{u_i} = adyacentes de v_i

- PSEUDO CÓDIGO (OPTIMIZACIÓN usando MIN HEAP)

- previamente se tendría que haber pasado una heap como parámetro, manteniendo los vértices ordenados por su d_v .
- Se reemplaza la línea (5) por vértice de menor distancia (H, u), el cual hará un deleteMin (H) y la guardará en u (no se realiza perc-down (reig)).
- Como línea 12, se agrega fuera del 2º bucle (7) el perc-down (reig), una vez actualizadas las distancias de los adyacentes.

- $T(n) = O(|E| \cdot \log |V|)$

- La operación vértice de menor distancia (H, u) es $O(\log |V|)$ y como se realiza $|V|$ veces $\rightarrow$ el costo de la operación es de $O(|V| \cdot \log |V|)$.
- El bucle (7) supone modificar la prioridad (distancia) y reorganizar la heap. Cada iteración es de $O(\log |V|) \rightarrow$ realiza $|E|$ iteraciones $\rightarrow O(|E| \log |V|)$.
- El costo TOTAL del algoritmo es $(|V| \log |V| + |E| \log |V|)$ es $O(|E| \log |V|)$

- PSEUDOCÓDIGO (variante de la Heap).

DIJKSTRA (G, w, s, H). {

// $d_v = v.dist$, $C_{u,v} = \text{coste arista } (u,v)$.

para $c/\text{vértice } v \in |V|$ hacer

| $d_v = \infty$; $p_v = 0$;

$d_s = 0$; insert-Heap(s, d_s);

para $c/\text{vértice } v \in |V|$ hacer

| delete-Min(H, u);

| mientras (u) esté tratado hacer

| | delete-Min(H, u);

| $u.con = \text{TRUE}$;

| para $c/\text{adyacente } (w)$ al vértice (u) hacer

| | si $(w.con = \text{FALSE})$ entonces

| | | si $(w.dist > u.dist + C_{u,w})$ entonces

| | | $w.dist = u.dist + C_{u,w}$;

| | | Insert-Heap($w, w.dist$);

| }

• La variación es que se inserta $c/\text{adyacente } w$ y su d_w , y/o que se modifica.

$$T(n) = O(|E| \log |V|)$$

• El tamaño de la heap puede crecer hasta $|E|$.

Dado que $|E| \leq |V|^2 \rightarrow \log |E| \leq 2 \log |V|$, el costo total no VARIA.

$$T(|V|, |E|) = \sum_{i=1}^{|V|} (a_i \cdot \log |E| + c_1 + a_{u_i} (c_2 + \log |E|)) = \text{ya que } \log |E| = 2 \log |V|$$

$$= |E| \cdot \log |E| + |V| c_1 + |E| c_2 + |E| \log |E| = \frac{4}{3} |E| \log |V| + c_1 |V| + c_2 |E|$$

$$T(|V|, |E|) = O(|E| \log |V|)$$

Algoritmos de costo mínimo p/ aristas NEGATIVAS

- Se lleva a cabo ~~en~~ usando una cola auxiliar que vaya almacenando los adyacentes.
- Los vértices guardan la misma info que antes, excepto el campo de "conado".

PSEUDOCÓDIGO

PESONEGATIVO(G, w, s) {

| $q = \text{cola de adyacentes}$;

| para $c/\text{vértice } (v)$ hacer

| | $d_v = \infty$;

| $d_s = 0$;

| $q.\text{encolar}(s)$;

| mientras $\text{colaNoVacia}(q)$ hacer

| | Vértice $v = q.\text{desencolar}()$;

```

para c/ vértice (w) adyacente a (v), hacer
    si (  $d_w > d_v + C_{v,w}$  ) entonces
         $d_w = d_v + C_{v,w}$ ;
         $p_w = v$ ;
    si ( w no está en q ) entonces
        q. encolar (w);
}

```

• $T(n)$

• Dado que c/ vértice puede ser desencolado como máximo $|V|$ veces, el

$$T(|E|, |V|) = O(|E| \cdot |V|)$$

• Caminos mínimos p/ GRAFOS ACÍCLICOS (Optimización del algoritmo de Dijkstra).

- Mediante el orden topológico.
- La selección de cada vértice se realiza siguiendo el orden topológico.
- Ésta estrategia funciona correctamente, dado que al seleccionar un vértice v , no se va a encontrar una distancia d_v menor, porque ya se procesaron todos los caminos que llegan a él.

- Costo Total del algoritmo: $T(|V|, |E|) = O(|E| + |V|)$

• Caminos mínimos p/ todos los pares de vértices.

- Algoritmo de Floyd ($O(|V|^3)$)

- Se utilizan 2 matrices de dimensión $|V| \times |V|$:

• una matriz D de costos mínimos.

• una matriz P de vértices intermedios (son los que se recorren para ir desde el origen (s) hasta el final (t)).

- PSEUDOCÓDIGO

```

para k=1 hasta cant.Vértices(G) hacer
    para i=1 hasta cant.Vértices(G) hacer
        para j=1 hasta cant.Vértices(G) hacer
            si (  $D[i,j] > D[i,k] + D[k,j]$  ) entonces
                 $D[i,j] = D[i,k] + D[k,j]$ ;
                 $P[i,j] = k$ ;

```

- Ver página 426 del libro.

* Árbol de expansión mínima (AEM)

- Estos algoritmos se realizan sobre GRAFOS NO DIRIGIDOS y CONEXOS

Definición:

El árbol de expansión mínima de un grafo (G) NO DIRIGIDO y CONEXO, es un árbol formado por las aristas de G que conectan TODAS las vértices con un costo total mínimo.

PRIM: - El árbol crece por etapas.

- En cada etapa se:

- elige un vértice como raíz.

- le agrega al árbol una arista y un vértice asociado.

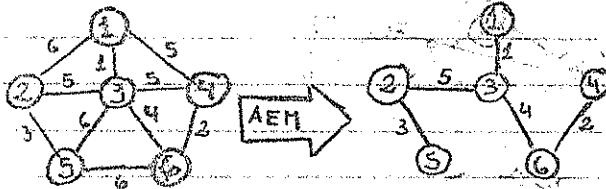
- Se seleccionan aquellas aristas (u,v) de mínimo costo que amplie:

el árbol, y no el árbol.

para evitar los ciclos.

- El número de aristas de un AEM es $|V| - 1$.

EJEMPLO



- La implementación es muy similar al algoritmo de Dijkstra. Sólo cambia la forma de ACTUALIZAR LAS DISTANCIAS: se actualiza D_w con $C_{v,w}$ si $D_w > C_{v,w}$.

- Ahora d_v es el peso del arista mínima que conecta v con algún vértice conocido y p_v es el último vértice en conectar v en cambio en d_v .

- EJEMPLO

V	con	d_v	p_v	V	con	d_v	p_v	V	con	d_v	p_v	V	con	d_v	p_v	V	con	d_v	p_v	V	con	d_v	p_v	V	con	d_v	p_v	V	con	d_v	p_v
1	F	0	0	1	T	0	0	1	T	0	0	1	T	0	0	1	T	0	0	1	T	0	0	1	T	0	0	1	T	0	0
2	F	∞	0	2	F	6	1	2	F	5	3	2	F	5	3	2	F	5	3	2	F	5	3	2	F	5	3	2	F	5	3
3	F	∞	0	3	F	1	1	3	T	1	1	3	T	1	1	3	T	1	1	3	T	1	1	3	T	1	1	3	T	1	1
4	F	∞	0	4	F	5	1	4	F	5	1	4	F	2	6	4	T	2	6	4	T	2	6	4	T	2	6	4	T	2	6
5	F	∞	0	5	F	∞	0	5	F	6	3	5	F	6	3	5	F	3	2	5	T	3	2	5	T	3	2	5	T	3	2
6	F	∞	0	6	F	∞	0	6	F	4	3	6	T	4	3	6	T	4	3	6	T	4	3	6	T	4	3	6	T	4	3

- Orden de Ejecución

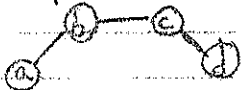
• sin usar minHeap $\rightarrow T(V, E) = O(V^2)$

• usando minHeap $\rightarrow T(V, E) = O(E \log V)$

ALGORITMO DE KRUSKAL ($O(|E| \log |V|)$)

- Se va seleccionando las aristas en orden creciente según su peso y las acepta si no originan un ciclo. SIEMPRE SE EVITAN LOS CICLOS (sino no sería un árbol).
- Cada vértice se encuentra ordenado en una minHeap, p/ recuperar el costo mínimo en c/paso.
- Inicialmente c/ vértice pertenece a su propio conjunto (en árbol de una componente CONEXA) $\rightarrow |V|$ conjuntos con un único elemento.
- Se intenta agregando aristas hasta lograr que todos los vértices formen un AEM (una sola componente conexa).
- Al aceptar una arista se realiza la unión de 2 conjuntos (componentes conexas).
- Si los vértices (u) y (v) están en el mismo conjunto, la arista (u,v) es RECHAZADA porque al aceptarla formarían un ciclo.
- En c/ punto del proceso, 2 vértices pertenecen al mismo conjunto. Si están conectados.

Componente ①



"a está conectado con b."
"a está conectado con c"
"a está conectado con d"
... (etc.)

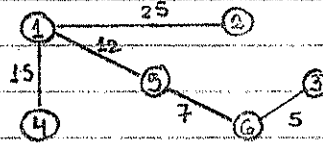
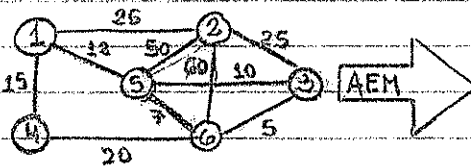
Componente ②

③

Unir la componente ① con la ② mediante la arista (a,c) , originaría un ciclo:



EJEMPLO

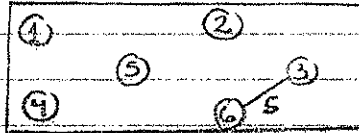


* Tenemos las aristas: $(3,6); (5,6); (5,3); (1,5); (1,4); (4,6); (1,2); (2,3); (5,2); (2,6)$.

* Tenemos los conjuntos (componentes conexas): ①, ②, ③, ④, ⑤, ⑥

1^{er} etapa:

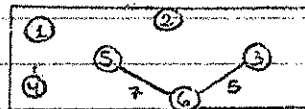
ARISTA: $(3,6)$; se unen los conjuntos ③ y ⑥.



2^{da} ETAPA:

ARISTA: $(5,6)$; se unen las componentes ⑤ y ⑥-3

ya que ⑥ $\in$ Conjunto y ⑤ $\notin$ conjunto



3^{er} etapa: ARISTA: $(5,3)$; No se agrega esta arista porque causaría el ciclo $(5-3-6-5)$. El gráfico quedaría igual.
ya q' ⑤ $\in$ Conjunto y ③ $\in$ Conjunto.

4^{ta} etapa: ARISTA: $(1,5)$; se unen componentes ① y ⑤-6-3.
ya que ⑤ $\in$ Conjunto y ① $\notin$ Conjunto.

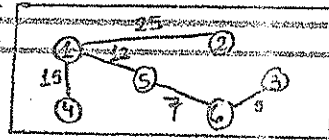


5^{ta} etapa: ARISTA: $(1,4)$; se unen ④ y ①-5-6-3
ya que ② $\in$ Conjunto y ④ $\notin$ Conjunto.



6^{ta} etapa: ARISTA: $(4,6)$; No se agrega porque causaría el ciclo $(4-6-5-3-4)$. El gráfico quedaría igual.
ya que ④ $\in$ Conjunto y ⑥ $\in$ Conjunto.

7^a etapa: ARISTA: (1,2) ; se unen ② y ④-1-5-6-3
ya que ① ∈ Conjunto y ② ∉ Conjunto.



- El procedimiento finaliza cuando se tienen $|V|-1$ aristas agregadas.

- Tiempo de Ejecución $T(|V|, |E|) = O(|E| \log |V|)$

- Se organizan las aristas en una minheap para optimizar la recuperación del mínimo.
- El tamaño de la minheap es $|E|$ y extraer ϕ arista es de $O(\log |E|)$.
→ extraer todos los vértices en el peor de los casos es $O(|E| \log |E|)$.
- Como $|E| \leq |V|^2$, $\log |E| \leq \log |V|^2 = 2 \log |V|$.
→ el coste Total es de $O(|E| \log |V|)$.

